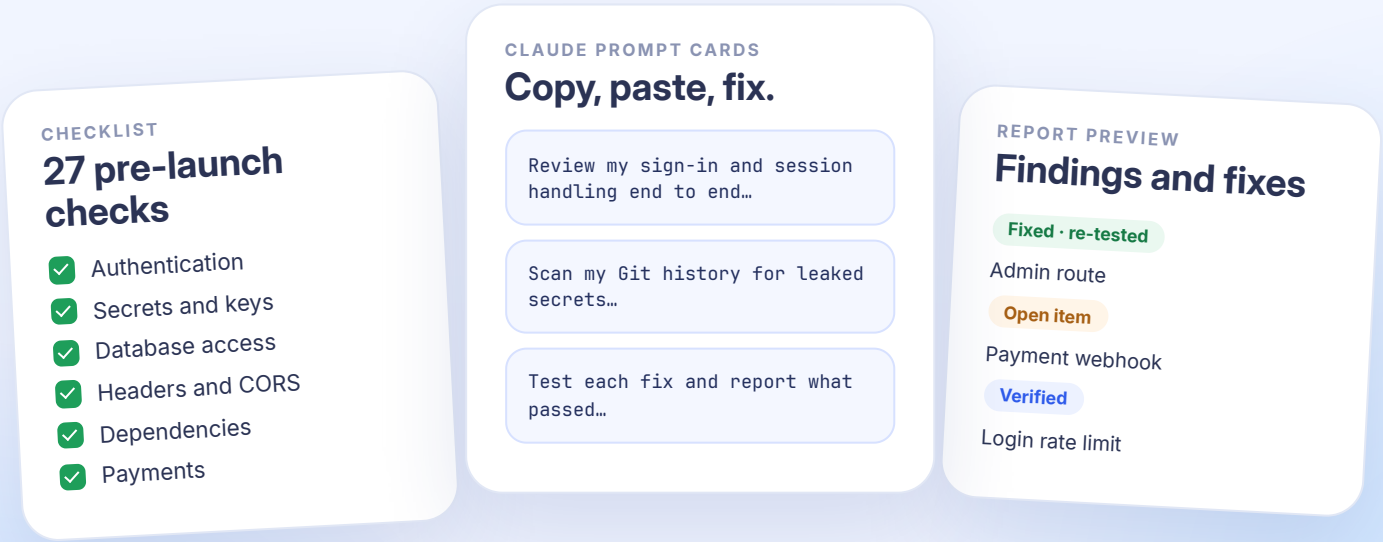
PRE-LAUNCH SECURITY TOOLKIT

Lock down your vibe-coded app before launch.

27 security checks, a detailed Claude prompt for each, quick self-tests, and a final test-and-report workflow, written for people who build with AI and want to ship safely.

27	36	7	1
pre-launch checks	copy-paste prompts	bonus checks	final security report



27/27 SECURITY PASS

Works with Claude Code, Cursor, Lovable, Bolt, v0 & Replit

INSIDE THIS TOOLKIT

Contents

27 checks in the same order as the video, each with a plain-English explanation, a 2-minute self-test, a detailed copy-paste prompt and a “done when” list. Everything is clickable.

– Read this first

– Your first hour & red flags

– The 27-check checklist

00 Project context + map your app

B Bonus checks (B1–B7)

! Emergency card: if something goes wrong

– Keep it secure after launch

– What your final report looks like

– Glossary

The 27 checks

01 Set up proper authentication CRITICAL

02 Hide API keys CRITICAL

03 Secure my database CRITICAL

04 Enforce HTTPS HIGH

05 Hash passwords CRITICAL

06 Validate my forms server-side HIGH

07 Protect against XSS HIGH

08 Log and monitor suspicious activity MEDIUM

09 Lock down my admin routes CRITICAL

10 Use parameterized queries CRITICAL

11 Check my environment variables CRITICAL

12 Add rate limiting HIGH

13 Restrict my CORS settings MEDIUM

14 Add security headers MEDIUM

15 Secure my session cookies HIGH

16 Add CSRF protection HIGH

17 Make password-reset links expire HIGH

18 Restrict file upload types and sizes HIGH

19 Verify prices and payments CRITICAL

20 Disable production debug mode HIGH

21 Update vulnerable dependencies HIGH

22 Remove unused packages MEDIUM

23 Check for exposed files CRITICAL

24 Scan my Git history for leaked secrets CRITICAL

25 Revoke and replace compromised keys CRITICAL

26 Run a thorough security review REVIEW

27 Test the protections and give me a report VERIFY

START HERE

You built it fast. Now make it safe.

AI tools are brilliant at building features that work. They are much less reliable at building features that are **safe**, because nothing looks broken when a security check is missing. This toolkit closes that gap: you don't need to be a security expert, you just need to know what to ask, and how to check the answer.

How to use this toolkit

1

Save a checkpoint

Commit your code (or duplicate the project in Lovable/Bolt/Replit). Work on a branch. If anything breaks, you can roll back.

2

Map your app

Fill in the Project Context block, then run Prompt 00. Claude creates a map of every route, key and data store.

3

Work through the checks

Run checks 01–25 one at a time. Do the 2-minute self-check, paste the prompt, review the changes, tick "Done when".

4

Review, test, report

Run Check 26 for a full review and Check 27 to prove every fix with tests and get your security report.

How each check is laid out

WHAT THIS MEANS

The idea in plain English, no jargon.

WHAT GOES WRONG

A realistic attack on **Plantly**, the demo app from the video.

2-MINUTE SELF-CHECK

A quick test you can run yourself, before and after.

CLAUDE PROMPT

A detailed, copy-paste prompt: audit → fix → verify → report.

DONE WHEN

Clear finish line. Don't move on until every box is true.

WATCH OUT FOR

Common traps and stack-specific gotchas.

Tips for getting great results from the prompts

- **Give the AI your whole codebase.** These prompts work best in tools that can read and edit files: Claude Code, Cursor, Windsurf, or the chat inside Lovable, Bolt, v0 or Replit.
- **One check per conversation.** Start fresh for each check, and paste your Project Context first. Focused sessions give far better results.
- **Read the findings table before accepting changes.** If something doesn't make sense, ask "why?" or "show me the code that proves this".
- **Never paste real secrets into a chat.** Refer to keys by their variable name. The prompts already tell Claude to do the same.
- **Test after every check.** Click through login, checkout and your main features. Security fixes occasionally break things; better to find out now.
- **Copying prompts:** long prompts span pages. For clean copying, use the companion `SECURITY_PROMPTS.md` file, which has every prompt as plain text.

An honest note

AI reviewers can miss real problems and occasionally invent fake ones. This toolkit dramatically raises your baseline, but it isn't a guarantee. If you handle payments, health data, children's data or large numbers of users, also get a professional security review or penetration test before launch.

SHORT ON TIME?

Your first hour: stop the worst leaks.

If you're launching soon and can only do a little, do these eight in this order. They cover the issues that most often lead to stolen data, drained API credits and surprise bills in AI-built apps.

THE PRIORITY EIGHT

02	**Hide API keys**
11	**Environment variables**
24	**Scan Git history**
25	**Rotate leaked keys**
03	**Secure the database**
01	**Server-side auth**
09	**Lock admin routes**
19	**Prices & payments**

Then work through the rest in order and finish with 26 and 27.

RED FLAGS: DROP EVERYTHING IF ANY IS TRUE

- Your OpenAI/Stripe/other secret key appears anywhere in front-end code → 02 · 25
- You use Supabase or Firebase and never wrote RLS policies or security rules → 03
- Changing an ID in the URL shows someone else's data → 03
- Your admin page is "protected" by hiding the link → 09
- Your app trusts a price or total sent from the browser → 19
- Orders/plans unlock on the "success" page instead of a webhook → 19
- You once committed a `.env` file, even briefly → 24 · 25
- `https://yourapp.com/.env` returns anything except 404 → 23 · 25
- Your AI endpoint has no login requirement or usage limit → 12 · B5
- You see stack traces or SQL errors on the live site → 20

Already live with real users?

Make changes on a branch or preview deployment, test them, then ship. Some fixes (rotating session secrets, stricter cookies, new rate limits) can log users out or block legitimate traffic. The prompts tell Claude to ask before making changes users would notice, so read those questions carefully.

PRINT THIS PAGE

The 27-check pre-launch checklist

Grouped by area so you can see your coverage at a glance. Numbers match the video.

	#	CHECK	SEVERITY	NOTES / DATE DONE
IDENTITY & ACCESS				
☐	01	Set up proper authentication	CRITICAL	
☐	05	Hash passwords	CRITICAL	
☐	09	Lock down my admin routes	CRITICAL	
☐	15	Secure my session cookies	HIGH	
☐	17	Make password-reset links expire	HIGH	
SECRETS & KEYS				
☐	02	Hide API keys	CRITICAL	
☐	11	Check my environment variables	CRITICAL	
☐	24	Scan my Git history for leaked secrets	CRITICAL	
☐	25	Revoke and replace compromised keys	CRITICAL	
DATA & INPUTS				
☐	03	Secure my database	CRITICAL	
☐	06	Validate my forms server-side	HIGH	
☐	07	Protect against XSS	HIGH	
☐	10	Use parameterized queries	CRITICAL	
☐	18	Restrict file upload types and sizes	HIGH	
☐	19	Verify prices and payments	CRITICAL	
NETWORK & BROWSER				
☐	04	Enforce HTTPS	HIGH	
☐	12	Add rate limiting	HIGH	
☐	13	Restrict my CORS settings	MEDIUM	
☐	14	Add security headers	MEDIUM	
☐	16	Add CSRF protection	HIGH	
DEPLOYMENT & SUPPLY CHAIN				
☐	20	Disable production debug mode	HIGH	
☐	21	Update vulnerable dependencies	HIGH	
☐	22	Remove unused packages	MEDIUM	
☐	23	Check for exposed files	CRITICAL	
MONITORING & RESPONSE				
☐	08	Log and monitor suspicious activity	MEDIUM	
☐	26	Run a thorough security review	REVIEW	
☐	27	Test the protections and give me a report	VERIFY	

00

SETUP · 10 MIN · DO THIS FIRST

Project context + map your app

Give Claude the big picture once, so every check after this is sharper.

Every prompt in this toolkit works on its own, but it works **much** better when the AI knows your stack. Fill in the block below once, save it somewhere handy, and paste it at the top of every new conversation, before the check's prompt.

> YOUR PROJECT CONTEXT — Fill in the brackets once · reuse everywhere

```
PROJECT CONTEXT (paste this above any prompt in this toolkit)
- What the app does: [e.g. Plantly – people track and order houseplants]
- Who uses it: [public consumers / my team / businesses]
- Built with: [Lovable / Bolt / v0 / Cursor / Claude Code / Replit / other]
- Stack: [e.g. Next.js 15 + Supabase + Stripe + OpenAI]
- Hosting: [Vercel / Netlify / Render / Railway / Fly / VPS / other]
- Auth: [Supabase Auth / Clerk / Auth.js / Firebase / custom]
- User roles: [e.g. user, admin]
- Sensitive data: [emails, addresses, payments, health, messages, files…]
- Status: [not launched yet / live with ___ users]
- Anything I'm worried about: [optional]
```

Then run Prompt 00. It creates `SECURITY_MAP.md`, a single map of every route, key, data store and risky feature in your app. Later prompts (especially 26) use it, and it's a great document to keep up to date.

> CLAUDE PROMPT · 00 · MAP MY APP — Run once at the start

```
Act as a senior application security engineer. Before we change anything, I want you to fully understand
this app's security surface. Inspect the whole codebase. Do NOT modify any application code in this
step.

Create a file called SECURITY_MAP.md containing:
1. Stack: languages, frameworks and versions, package manager, hosting/deployment target.
2. Data: the database(s) and how they are reached (server-only, or directly from the browser via
Supabase/Firebase SDKs), every table/collection and how sensitive its data is.
3. Auth: the provider or method, where the session lives, where checks happen, and the user roles that
exist.
4. Entry points: a table of EVERY page, API route, server action, edge/serverless function, webhook and
cron job, with: path · method · purpose · requires login? · roles allowed · inputs accepted.
5. Third-party services: each one, what it's used for, which key it uses, and whether that key is used
client-side or server-side.
6. Environment variables: names only (NEVER values), where each is used, and public vs secret.
7. Risky features: file uploads, payments, emails/SMS, AI/LLM calls, anything that fetches URLs, admin
tools, imports/exports.
8. Questions: anything you couldn't determine from the code that I need to answer.

Then give me your TOP 5 security concerns, ranked, in plain English, each with the check number from
this toolkit that addresses it. No fixes yet.
```

01

IDENTITY & ACCESS | CRITICAL | 20–40 MIN

01/27 SECURITY PASS

Set up proper authentication

Prove who every user is, on the server, on every single request.

WHAT THIS MEANS

Authentication is how your app knows who is making a request. The important part is **where** that check happens: a login screen and a hidden menu are just UI. The real check has to run on your server (or in your database rules) every time data is read or changed.

WHAT GOES WRONG

Plantly's dashboard only shows plants after login, because the React page checks `if (user)`. But `/api/plants` never checks anything, so anyone who copies that URL into a terminal gets every user's plants. The login screen was a curtain, not a lock.

2-MINUTE SELF-CHECK

- Log in, open DevTools → Network, right-click an API request → **Copy as cURL**. Delete the `cookie` / `authorization` line and run it. You should get **401**, not data.
- Open a protected page in a private window while logged out. It should redirect to login *and* its data request should fail.
- Change an ID in a URL or request body (e.g. `/api/users/42` → `/api/users/43`). You should never see someone else's data.

> CLAUDE PROMPT · CHECK 01

Paste your Project Context first, then this

```
Act as a senior application security engineer. Review how authentication works in this app from end to end and make it launch-ready.

AUDIT
1. Identify the auth method (e.g. Supabase Auth, Clerk, Auth.js/NextAuth, Firebase Auth, custom JWT, server sessions). If it is hand-rolled, tell me, and say whether I should move to a proven provider instead.
2. List EVERY server entry point: API routes, server actions, route handlers, edge/serverless functions, GraphQL resolvers, RPC/database functions, webhooks and cron routes. For each one state: does it need a signed-in user, and is that checked ON THE SERVER inside the handler itself (not only in the UI, a client-side route guard, or middleware alone)?
3. Confirm the server verifies identity properly on every request: sessions are looked up server-side, or JWTs are verified with a library (signature, expiry, issuer, audience). Flag any token that is decoded without being verified, any acceptance of alg "none", and any hard-coded or weak signing secret.
4. Confirm the user ID used to read or write data always comes from the verified session/token, never from the request body, query string, URL, or a header the client controls.
5. Check sign-up and login: generic errors ("Invalid email or password") that don't reveal whether an account exists, email verification before sensitive actions, sessions invalidated on logout and on password change, and a sensible session lifetime.
6. If OAuth/social login or magic links exist: the state parameter is used, callback/redirect URLs are an exact allowlist, and any ?next= / ?redirect= / ?returnTo= parameter only allows relative paths on my own site (no open redirects).

FIX
- Create one reusable server-side helper (e.g. requireUser() / getCurrentUser()) and call it at the top of every protected handler. Return 401 when there is no valid session.
- Remove any security decision that relies only on client-side code.
- For Supabase on the server, use the method that re-validates the user with the auth server (getUser / getClaims), not one that trusts the cookie as-is.
```

```
VERIFY
For every protected endpoint, give me a curl command with no credentials that should now return 401,
plus one that tries to read another user's resource and should return 403 or 404.

HOW TO WORK
- Audit first, and show me what you found before making large changes.
- Fix Critical and High issues directly. Ask me before any change users would notice (logging people
out, blocking a feature, changing a flow).
- Fix the root cause with the smallest safe change. Don't remove features, weaken other protections, or
leave "TODO: secure later" comments.
- If you add a package, name it, say why, and confirm it is a real, well-maintained package (not a look-
alike name).
- Never print real secret values; refer to them by variable name.
- If something lives outside the code (dashboards, hosting settings), give me exact click-by-click steps
instead.

FINISH WITH
1. Findings table: issue · file:line · severity · status (Fixed / Needs my action / Not applicable)
2. Files changed
3. How I can verify it myself in under 5 minutes
```

DONE WHEN

- Every protected endpoint returns 401 without a valid session.
- The user ID always comes from the verified session, never from the request.
- Logout and password change kill existing sessions.
- Login/sign-up errors don’t reveal whether an email is registered.
- Redirect parameters only allow your own paths.

WATCH OUT FOR

- **Middleware is not enough.** In 2025 a Next.js flaw let attackers skip middleware entirely. Check auth inside each route handler / server action too.
- **Next.js server actions are public endpoints.** Anyone can call them directly, so each one needs its own auth check.
- **Don’t roll your own.** Password storage, sessions and token handling are easy to get subtly wrong. Use Supabase Auth, Clerk, Auth.js, Firebase Auth or your framework’s built-in system.

02

SECRETS & KEYS | CRITICAL | 15–30 MIN

02/27 SECURITY PASS

Hide API keys

Anything your browser can see, every visitor can see. Secret keys live on the server only.

WHAT THIS MEANS

Code that runs in the browser is shipped to every visitor, in full. If a secret key (OpenAI, Anthropic, Stripe secret, SendGrid, Twilio, a database admin key) appears anywhere in front-end code, it is public. The fix is a small server route: the browser calls *your* server, your server adds the key and calls the third party.

WHAT GOES WRONG

Plantly's `app.js` calls the weather API with `?key=…`. A visitor opens DevTools, copies the key and uses it for their own app. Worse: an AI key copied by a bot can run up thousands of dollars of usage overnight, and the bill is yours.

2-MINUTE SELF-CHECK

- Run a production build, then search the output folder (`dist/` , `build/` , `.next/static/`) for `sk_` , `sk-` , `secret` , `Bearer` , `api_key` , `service_role` .
- On your live site, open DevTools → Sources, press Ctrl/Cmd+Shift+F and search the same words.
- Open DevTools → Network and click around. Look at the request URLs and headers: do any contain a key?

> CLAUDE PROMPT · CHECK 02

Paste your Project Context first, then this

```
Act as a senior application security engineer. Find every API key, token or credential that could reach the browser (or a mobile app bundle) and move every secret to the server.

AUDIT
1. Search the whole codebase, including config files, for hard-coded keys and tokens (patterns like sk_, sk-, pk_, AKIA, ghp_, xox, Bearer, api_key, apiKey, secret, token, password, service_role, private_key, and long random-looking strings).
2. List every third-party service the app talks to. For each: which key it uses, whether that key is called from client-side or server-side code, and whether the key is PUBLISHABLE (designed to be public) or SECRET.
   - Usually publishable: Stripe pk_ keys, the Supabase anon/publishable key (only safe if Row Level Security is on), Firebase web config, Google Maps browser keys restricted by domain.
   - Always secret: Stripe sk_/rk_ keys, OpenAI/Anthropic/other AI keys, Supabase service_role/secret key, database URLs, email/SMS provider keys, webhook secrets, JWT secrets, cloud credentials.
3. Check every environment variable that is exposed to the browser through a public prefix (NEXT_PUBLIC_, VITE_, REACT_APP_, EXPO_PUBLIC_, PUBLIC_, NUXT_PUBLIC_) and flag any that hold a secret.
4. Build the app for production and search the build output for secrets, so we know what actually ships.

FIX
- Move every secret into server-only environment variables.
- Replace each client-side call that needs a secret with a server route / server action / edge function that: requires a signed-in user where appropriate, validates its input, is rate-limited, and only performs the one specific operation the UI needs. Do NOT build a generic proxy that forwards any URL or any request body (that just turns my server into an open relay).
- For publishable keys that stay public, tell me which dashboard restrictions to add (allowed domains/referrers, allowed APIs, read-only scopes).

IMPORTANT
Removing a key from the code does not un-leak it. Give me a list of every secret that has EVER been in
```

```
client code or a public repo, so I can rotate it (Check 25).

HOW TO WORK
- Audit first, and show me what you found before making large changes.
- Fix Critical and High issues directly. Ask me before any change users would notice (logging people
out, blocking a feature, changing a flow).
- Fix the root cause with the smallest safe change. Don't remove features, weaken other protections, or
leave "TODO: secure later" comments.
- If you add a package, name it, say why, and confirm it is a real, well-maintained package (not a look-
alike name).
- Never print real secret values; refer to them by variable name.
- If something lives outside the code (dashboards, hosting settings), give me exact click-by-click steps
instead.

FINISH WITH
1. Findings table: issue · file:line · severity · status (Fixed / Needs my action / Not applicable)
2. Files changed
3. How I can verify it myself in under 5 minutes
```

DONE WHEN

- A search of the production build finds no secret keys.
- No secret uses a public env prefix.
- All third-party calls that need a secret go through your own server.
- Publishable keys have domain/scope restrictions set in their dashboards.
- You have a list of keys to rotate.

WATCH OUT FOR

- **“It’s minified” is not hiding.** Bundled JS is trivially searchable.
- **Mobile apps count too.** Keys inside an iOS/Android/Expo bundle can be extracted.
- **Lovable, Bolt and v0 projects** often put keys in front-end code during prototyping. Search for them specifically before launch.

03

DATA & INPUTS | CRITICAL | 30–60 MIN

03/27 SECURITY PASS

Secure my database

Users can read and change only their own data, even if they talk to the database directly.

WHAT THIS MEANS

Two things must be true. First, every query is scoped to the right user: Ana can fetch order #18 because it's hers, never order #22. Second, the database itself is locked down. If your front end talks straight to the database (Supabase, Firebase), the database rules **are** your security. The public key in your front end lets anyone query your tables, so the rules decide what they get.

WHAT GOES WRONG

Ana changes `/orders/18` to `/orders/22` and sees Ben's address. That's called an IDOR, and it's one of the most common bugs in real apps. Or: a Supabase table was created without Row Level Security, and anyone with the public anon key can download the whole `users` table.

2-MINUTE SELF-CHECK

- Create two test accounts. As account A, copy a request that loads A's data, swap in an ID belonging to B, and send it. Expect 403/404.
- **Supabase:** open Table Editor and confirm every table shows RLS enabled. Run **Advisors → Security Advisor** and fix every warning.
- **Firebase:** your rules must not contain `allow read, write: if true` or a date-based "test mode" rule.

> CLAUDE PROMPT · CHECK 03

Paste your Project Context first, then this

```
Act as a senior application security engineer specialising in database security. Make sure every user
can only read and change the data they are allowed to, and that the database itself is locked down.

AUDIT
1. List every table/collection and storage bucket. For each, describe who should be able to read,
create, update and delete rows (owner only, team members, admins, public).
2. Work out how the app reaches the database: only from server code, or directly from the browser
(Supabase client, Firebase SDK, etc.).
3. If the browser talks to the database directly:
   - Supabase/Postgres: Row Level Security must be ENABLED on every table in exposed schemas. Write
separate policies for SELECT, INSERT, UPDATE and DELETE based on auth.uid(). INSERT and UPDATE policies
need WITH CHECK so users can't create rows owned by someone else or change owner_id/user_id. Check views
(they bypass RLS unless created with security_invoker = true) and SECURITY DEFINER functions (they
bypass RLS, so they must check permissions themselves). Never base policies on user_metadata, because
users can edit it.
   - Firebase: rewrite rules per path so access depends on request.auth.uid and validates the fields
being written. Remove any test-mode or "if true" rules.
   - Storage buckets: private by default, with policies so users only access their own files.
4. If access goes through server code: check every query that fetches, updates or deletes by ID also
checks ownership/permission (look for IDOR). List endpoints where an ID comes from the client and the
query is not scoped to the current user.
5. Make sure the service_role/admin key and the database connection string are used ONLY in server code.
6. Check the database user's privileges (it shouldn't be a superuser), whether the database is reachable
from the public internet (IP allowlist, no 0.0.0.0/0), that connections use TLS, and that automated
backups are on.
7. Check API responses never return sensitive columns (password hashes, tokens, other users' emails,
internal flags). Select only the fields the UI needs.
```

```
FIX
Write the missing policies/rules and ownership checks. Show me every policy in full SQL/rules syntax
with a one-line plain-English explanation of each.

VERIFY
Write a test (or a script I can run) that uses two separate test users and proves user A cannot read,
update or delete user B's rows or files, and that a signed-out visitor using only the public key gets
nothing private.

HOW TO WORK
- Audit first, and show me what you found before making large changes.
- Fix Critical and High issues directly. Ask me before any change users would notice (logging people
out, blocking a feature, changing a flow).
- Fix the root cause with the smallest safe change. Don't remove features, weaken other protections, or
leave "TODO: secure later" comments.
- If you add a package, name it, say why, and confirm it is a real, well-maintained package (not a look-
alike name).
- Never print real secret values; refer to them by variable name.
- If something lives outside the code (dashboards, hosting settings), give me exact click-by-click steps
instead.

FINISH WITH
1. Findings table: issue · file:line · severity · status (Fixed / Needs my action / Not applicable)
2. Files changed
3. How I can verify it myself in under 5 minutes
```

DONE WHEN

- RLS / security rules are enabled on every table, collection and bucket.
- User A cannot read, edit or delete user B's data (tested, not assumed).
- Admin/service keys appear only in server code.
- The database isn't open to the whole internet, and backups are on.
- API responses never include password hashes or other users' private fields.

WATCH OUT FOR

- **RLS enabled with no policies** blocks everything; **RLS disabled** allows everything. Neither is what you want.
- **Supabase views and SECURITY DEFINER functions** are the classic RLS bypass.
- **MongoDB Atlas** network access set to `0.0.0.0/0` plus a weak password is a common breach.

04

NETWORK & BROWSER | HIGH | 10–20 MIN

04/27 SECURITY PASS

Enforce HTTPS

Every byte between your users and your servers is encrypted, with no plain-HTTP side doors.

WHAT THIS MEANS

HTTPS encrypts traffic so nobody on the same Wi-Fi, ISP or network can read or change it. Most hosts (Vercel, Netlify, Cloudflare, Render) give you HTTPS automatically, but you still need: HTTP redirected to HTTPS, an HSTS header so browsers never try HTTP again, no insecure `http://` links inside the app, and encrypted connections to your database and other services.

WHAT GOES WRONG

A user logs in at a café. One old API call in Plantly still uses `http://`, so their session token travels in plain text and anyone on the network can grab it. Or a custom domain on a VPS was never given a certificate, and the login form posts over HTTP.

2-MINUTE SELF-CHECK

- Visit `http://yourdomain.com` (and the `www.` version). It should instantly redirect to `https://`.
- Run your domain through **SSL Labs** (ssllabs.com/ssltest). Aim for an A.
- Open DevTools → Console on every main page and look for “Mixed Content” warnings.

> CLAUDE PROMPT · CHECK 04

Paste your Project Context first, then this

```
Act as a senior application security engineer. Make sure this app is HTTPS-only everywhere, with no plain-HTTP paths left.

AUDIT
1. Work out where the app is hosted and how TLS is handled (platform-managed, Cloudflare, nginx/Caddy on a VPS, load balancer).
2. Check that every HTTP request (apex domain, www, all subdomains, all paths) is permanently redirected (301/308) to HTTPS.
3. Search the codebase for hard-coded "http://" URLs: API calls, images, scripts, fonts, iframes, webhooks, OAuth callback URLs, email links, redirects, and environment variable values. localhost in development config is fine.
4. Check outgoing connections from the server: database, Redis, email, third-party APIs. Database connection strings should require TLS (for example sslmode=require for Postgres).
5. Check cookies are marked Secure.
6. If the app runs behind a proxy or load balancer, confirm it reads the original protocol correctly (for example app.set('trust proxy', 1) in Express) so HTTPS detection and redirects work without loops.

FIX
- Add or confirm the HTTP→HTTPS redirect at the host/proxy level.
- Add the header: Strict-Transport-Security: max-age=63072000; includeSubDomains. Only add includeSubDomains if every subdomain serves HTTPS. Do NOT add "preload" unless I explicitly confirm, because it is hard to undo.
- Replace every insecure URL with https:// or a relative URL.
- Enforce TLS on database and service connections.

VERIFY
Give me curl commands that show the redirect and the HSTS header, and tell me how to check the browser console for mixed-content warnings.
```

```
HOW TO WORK
- Audit first, and show me what you found before making large changes.
- Fix Critical and High issues directly. Ask me before any change users would notice (logging people out, blocking a feature, changing a flow).
- Fix the root cause with the smallest safe change. Don't remove features, weaken other protections, or leave "TODO: secure later" comments.
- If you add a package, name it, say why, and confirm it is a real, well-maintained package (not a look-alike name).
- Never print real secret values; refer to them by variable name.
- If something lives outside the code (dashboards, hosting settings), give me exact click-by-click steps instead.

FINISH WITH
1. Findings table: issue · file:line · severity · status (Fixed / Needs my action / Not applicable)
2. Files changed
3. How I can verify it myself in under 5 minutes
```

DONE WHEN

- HTTP always redirects to HTTPS, on every domain and subdomain you use.
- The HSTS header is present on HTTPS responses.
- No mixed-content warnings in the browser console.
- Database and service connections use TLS.
- Cookies have the `Secure` flag.

WATCH OUT FOR

- **HSTS preload** is a near-permanent commitment. Skip it until everything is stable.
- **Old subdomains** (staging, blog, docs) are often forgotten and still serve HTTP.

05

IDENTITY & ACCESS | CRITICAL | 15–30 MIN

05/27 SECURITY PASS

Hash passwords

Store passwords so that even you, and anyone who steals your database, can't read them.

WHAT THIS MEANS

Passwords must be stored as a one-way, salted, deliberately slow hash using **Argon2id**, **scrypt** or **bcrypt**. "Encrypted" passwords can be decrypted, and fast hashes like MD5 or SHA-256 can be cracked by the billions per second. If you use an auth provider (Supabase, Clerk, Firebase, Auth0), it already does this. Your job is to make sure passwords don't leak *anywhere else*.

WHAT GOES WRONG

Plantly's database is leaked. Passwords are stored as plain text, so attackers try each email + password on Gmail, banks and PayPal. Because people reuse passwords, your bug becomes your users' stolen accounts everywhere.

2-MINUTE SELF-CHECK

- Look at your users table. Good values start like `$argon2id$`, `$2b$12$` or `$scrypt$`. Plain text, or a 32/64-character hex string, is bad.
- Search your logs and error tracker for a test password you just used. It should appear nowhere.
- Check that no API response, including `/api/me` and admin user lists, contains a `password` or `password_hash` field.

> CLAUDE PROMPT · CHECK 05

Paste your Project Context first, then this

```
Act as a senior application security engineer. Make sure passwords are handled and stored correctly
everywhere in this app.

AUDIT
1. Find every place passwords are handled: sign-up, login, change password, reset password, admin
"create user", invites, seed scripts, data imports and tests.
2. Determine how passwords are stored. If an auth provider handles it, confirm that this app does not
ALSO store, log, or forward passwords anywhere (its own users table, analytics, logs, emails, error
tracking).
3. If passwords are stored by this app, check the algorithm. Plain text, reversible encryption, MD5,
SHA-1, SHA-256 or any unsalted/fast hash is a Critical finding.
4. Check that passwords are compared with the library's verify function (constant-time), not with == on
hashes.
5. Check request logging, error handlers and error-tracking tools (Sentry etc.) to make sure request
bodies containing passwords are never recorded.
6. Check no API response or client state ever includes a password or password hash, and that passwords
are never stored in localStorage or cookies.

FIX
- Use Argon2id (OWASP minimum: 19 MiB memory, 2 iterations, parallelism 1) via a maintained library, or
bcrypt with cost 12 (minimum 10). Note bcrypt only uses the first 72 bytes of a password, so set a max
password length accordingly.
- If existing hashes are weak, migrate safely: on each user's next successful login, verify against the
old hash and immediately re-hash with the new algorithm. Tell me what to do about users who never log in
again (for example force a reset).
- Password policy (NIST-style): minimum 8 characters (12+ recommended), allow at least 64, allow all
characters including spaces, no forced "1 uppercase + 1 symbol" rules. Optionally block known-breached
passwords using the Have I Been Pwned range API (k-anonymity).
```

```
- Redact password fields from all logs and error reports.
- Never email a password to a user.

VERIFY
Show me what a stored hash looks like now, and give me a test that proves a correct password logs in, a
wrong one fails, and the plain password appears in no log output.

HOW TO WORK
- Audit first, and show me what you found before making large changes.
- Fix Critical and High issues directly. Ask me before any change users would notice (logging people
out, blocking a feature, changing a flow).
- Fix the root cause with the smallest safe change. Don't remove features, weaken other protections, or
leave "TODO: secure later" comments.
- If you add a package, name it, say why, and confirm it is a real, well-maintained package (not a look-
alike name).
- Never print real secret values; refer to them by variable name.
- If something lives outside the code (dashboards, hosting settings), give me exact click-by-click steps
instead.

FINISH WITH
1. Findings table: issue · file:line · severity · status (Fixed / Needs my action / Not applicable)
2. Files changed
3. How I can verify it myself in under 5 minutes
```

DONE WHEN

- Passwords are stored with Argon2id, scrypt or bcrypt (or fully by an auth provider).
- No password or hash appears in logs, error reports, emails or API responses.
- A migration path exists for any old, weak hashes.
- Password rules are length-based, not annoying symbol rules.

WATCH OUT FOR

- **Encryption is not hashing.** If you can decrypt it, so can an attacker who gets the key.
- **Request loggers** that log the full body will happily log passwords. Redact them.

06

DATA & INPUTS | HIGH | 20–45 MIN

06/27 SECURITY PASS

Validate my forms server-side

Front-end validation is for user experience. Server validation is for security.

WHAT THIS MEANS

Anything you check in the browser (required fields, min/max, dropdown options) can be skipped by sending the request directly. Your server must validate every input itself: type, length, range, format and allowed values. It should also accept **only the fields you expect**, so nobody can sneak in extras like `role` or `price`.

WHAT GOES WRONG

In the video, Plantly's order form sends `quantity: -3` straight to the server, which happily creates a negative total. Another classic: the "update profile" endpoint saves the whole request body, so a user adds `"role": "admin"` and promotes themselves.

2-MINUTE SELF-CHECK

- In DevTools → Network, right-click a form request → **Copy as fetch**, paste into the console, edit values (negative numbers, 100,000-character strings, wrong types) and send.
- Add an extra field such as `"role":"admin"` or `"credits":99999` to a profile update. It must be ignored or rejected.

> CLAUDE PROMPT · CHECK 06

Paste your Project Context first, then this

```
Act as a senior application security engineer. Add strict server-side validation to every input this app accepts.

AUDIT
1. List every endpoint, server action, form handler, webhook and background job that accepts input from outside: request body, query string, URL params, headers, cookies, uploaded file metadata. Put them in a table with the fields each one accepts.
2. For each, note whether it is validated on the server today, and how.
3. Look specifically for mass assignment: places where the request body (or a spread of it, like {...req.body} or **data) is passed straight into a database create/update. Flag any field that must be server-controlled (id, user_id/owner_id, role, is_admin, price, amount, status, credits, balance, plan, is_verified, email_verified, created_at).

FIX
- Use ONE validation library that fits the stack (for example Zod for TypeScript, Pydantic for Python) and validate at the very start of every handler, before any business logic or database call.
- For each field define: type, required/optional, min/max for numbers, max length for EVERY string, allowed values (enums) where possible, formats (email, URL, UUID, date), and integers in minor units (cents) for money.
- Reject unknown fields (strict mode) and copy only allowed fields into database writes. Server-controlled fields must never come from client input.
- Normalise inputs (trim whitespace, lowercase emails) before saving.
- Return a consistent 400 response with field-level messages, never a stack trace.
- Set a maximum request body size.
- Share the same schemas with the front end where possible so the rules stay in sync, but the server check is the one that counts.

VERIFY
Write tests that send: a negative quantity, a 100,000-character string, a wrong type, a missing required field, and an extra "role":"admin" field. Each must be rejected or ignored, and nothing invalid may reach the database.
```

```
HOW TO WORK
- Audit first, and show me what you found before making large changes.
- Fix Critical and High issues directly. Ask me before any change users would notice (logging people
out, blocking a feature, changing a flow).
- Fix the root cause with the smallest safe change. Don't remove features, weaken other protections, or
leave "TODO: secure later" comments.
- If you add a package, name it, say why, and confirm it is a real, well-maintained package (not a look-
alike name).
- Never print real secret values; refer to them by variable name.
- If something lives outside the code (dashboards, hosting settings), give me exact click-by-click steps
instead.

FINISH WITH
1. Findings table: issue · file:line · severity · status (Fixed / Needs my action / Not applicable)
2. Files changed
3. How I can verify it myself in under 5 minutes
```

DONE WHEN

- Every input-accepting endpoint validates on the server with a schema.
- Every string has a maximum length; every number has a range.
- Unknown fields are rejected and server-owned fields can't be set by clients.
- Invalid input returns a clean 400, never a crash or stack trace.

WATCH OUT FOR

- **TypeScript types don't exist at runtime.** A `quantity: number` type does nothing against `"quantity": "-3"`.
- **Validation ≠ safe output.** Valid text can still contain HTML. That's Check 07.

07

DATA & INPUTS | HIGH | 20–40 MIN

07/27 SECURITY PASS

Protect against XSS

User text is always shown as text, never run as code.

WHAT THIS MEANS

Cross-site scripting (XSS) happens when something a user typed gets inserted into a page as HTML, so the browser runs it. That script then acts as whoever is viewing the page: reading their data, stealing their session, or taking actions as them. Modern frameworks escape text by default; XSS sneaks in through the escape hatches.

WHAT GOES WRONG

Ben posts a comment containing a `<script>` tag. Plantly renders comments with `innerHTML`, so the script runs for everyone who reads the comment, including the admin. In the video, the fix is “encode on output”: the comment is shown as text instead of being run.

2-MINUTE SELF-CHECK

- In every field that’s displayed somewhere (name, bio, comment, title), enter `<img src=x onerror=alert(1)>`. If an alert pops up anywhere, you have XSS.
- In any URL/website field, enter `javascript:alert(1)` and click the resulting link.
- Search your code for `dangerouslySetInnerHTML`, `innerHTML`, `v-html`, `{@html`.

> CLAUDE PROMPT · CHECK 07

Paste your Project Context first, then this

```
Act as a senior application security engineer. Find and fix every place where user-controlled content could be executed as code in a browser (cross-site scripting).

AUDIT
1. Search for dangerous output sinks and list each with file:line and where its data comes from:
   - React: dangerouslySetInnerHTML. Vue: v-html. Svelte: {@html}. Angular: [innerHTML] or bypassSecurityTrust*.
   - DOM: innerHTML, outerHTML, insertAdjacentHTML, document.write, jQuery .html()/.append() with strings.
   - Server templates: |safe, mark_safe, autoescape off, Markup(), <%- %> in EJS, {{{ }}} in Handlebars, raw() helpers.
   - Code execution: eval, new Function, setTimeout/setInterval with a string.
2. Check user-controlled URLs placed in href, src, action or window.location. Only http:, https: and mailto: should be allowed; javascript: and data: must be blocked.
3. Check user data inserted inside <script> tags, inline event handlers, style attributes, or JSON embedded in HTML.
4. Check Markdown and rich-text rendering (does it allow raw HTML?), rich-text editors, SVG uploads, and HTML emails or PDFs generated with user content.
5. Treat AI/LLM output as untrusted user input too. If model output is rendered as HTML or Markdown, it must be sanitised.

FIX
- Prefer the framework's normal escaped rendering ({value} in JSX, {{ }} in templates).
- Where HTML is genuinely needed, sanitise with DOMPurify (browser) or a maintained server-side sanitiser, using a strict allowlist of tags and attributes.
- Validate URL schemes before rendering links.
- Make sure session cookies are HttpOnly (Check 15) and prepare for a Content-Security-Policy (Check 14) as a second line of defence.
```

```
VERIFY
Write tests that submit these payloads through each affected input and assert they render as harmless
text:
<img src=x onerror=alert(1)>
"><svg onload=alert(1)>
javascript:alert(1)

HOW TO WORK
- Audit first, and show me what you found before making large changes.
- Fix Critical and High issues directly. Ask me before any change users would notice (logging people
out, blocking a feature, changing a flow).
- Fix the root cause with the smallest safe change. Don't remove features, weaken other protections, or
leave "TODO: secure later" comments.
- If you add a package, name it, say why, and confirm it is a real, well-maintained package (not a look-
alike name).
- Never print real secret values; refer to them by variable name.
- If something lives outside the code (dashboards, hosting settings), give me exact click-by-click steps
instead.

FINISH WITH
1. Findings table: issue · file:line · severity · status (Fixed / Needs my action / Not applicable)
2. Files changed
3. How I can verify it myself in under 5 minutes
```

DONE WHEN

- No unsanitised `innerHTML` / `dangerouslySetInnerHTML` / `v-html` with user data.
- User-supplied links only allow http, https and mailto.
- Markdown/rich text and AI output are sanitised.
- XSS test payloads render as plain text everywhere.

WATCH OUT FOR

- **Admin panels are a favourite target.** Stored XSS that fires when an admin views user content can hand over the whole app.
- **Sanitise on output, not just on input.** Data can enter your database from imports, APIs or old versions of your app.

08

MONITORING & RESPONSE | MEDIUM | 20–40 MIN

08/27 SECURITY PASS

Log and monitor suspicious activity

If something bad happens, you find out in minutes, not months.

WHAT THIS MEANS

Logs are your security camera. You want a record of security-relevant events (logins, failures, permission denials, admin actions, payment events) and alerts when patterns look wrong. Equally important: logs must **not** contain secrets or passwords, or they become a breach of their own.

WHAT GOES WRONG

In the video, Ben's account gets five failed logins in two seconds. Without logging, nobody notices the attack until users complain or the data shows up for sale. Without timestamps and IPs, you can't even tell what was accessed.

2-MINUTE SELF-CHECK

- Fail a login five times. Can you find those events in your logs within two minutes, with time, account and IP?
- Would anyone be alerted? If the answer is "no", you have logs but no monitoring.
- Search your logs for `password`, `token`, `authorization`. They should be redacted.

> CLAUDE PROMPT · CHECK 08

Paste your Project Context first, then this

```
Act as a senior application security engineer. Add security logging and basic alerting to this app,
without logging anything sensitive.

AUDIT
1. Describe the current logging: what is logged, where it goes (console, Vercel/Netlify logs, Sentry,
Datadog, Axiom, Better Stack…), how long it is kept, and who can read it.
2. Check whether any log line or error report currently contains passwords, tokens, session IDs, API
keys, reset links, full card numbers, or full request bodies with personal data.

FIX
1. Add structured (JSON) security event logging through one small helper, e.g. logSecurityEvent(event,
details). Log these events:
   login success and failure, logout, sign-up, password change, password reset requested and completed,
email change, MFA changes, permission denied (401/403), admin actions and role changes, rate-limit hits
(429), payment and webhook events (including signature failures), file uploads, API key creation, and
unexpected 5xx errors.
2. Every event includes: UTC timestamp, event name, user ID (if known), IP address, user agent, request
ID, and outcome. Make the request ID visible in error responses so a user report can be matched to a log
line.
3. Redact sensitive values everywhere: passwords, tokens, cookies, Authorization headers, API keys, full
card data. Configure the error tracker to scrub them too. Strip newlines from user-supplied values
before logging to prevent log injection.
4. Set up alerts (or tell me exactly how to set them up in my provider) for: more than 10 failed logins
for one account or IP in 5 minutes, spikes in 401/403/429/5xx, any new admin or role change, and webhook
signature failures.
5. Recommend a retention period and make sure log access is limited to me/my team.

DELIVER
A one-page SECURITY_RUNBOOK.md: where the logs are, how to search for one user's activity, what each
alert means, and the first three things to do when one fires.
```

```
HOW TO WORK
- Audit first, and show me what you found before making large changes.
- Fix Critical and High issues directly. Ask me before any change users would notice (logging people
out, blocking a feature, changing a flow).
- Fix the root cause with the smallest safe change. Don't remove features, weaken other protections, or
leave "TODO: secure later" comments.
- If you add a package, name it, say why, and confirm it is a real, well-maintained package (not a look-
alike name).
- Never print real secret values; refer to them by variable name.
- If something lives outside the code (dashboards, hosting settings), give me exact click-by-click steps
instead.

FINISH WITH
1. Findings table: issue · file:line · severity · status (Fixed / Needs my action / Not applicable)
2. Files changed
3. How I can verify it myself in under 5 minutes
```

DONE WHEN

- Security events are logged with time, user, IP and request ID.
- No secrets or passwords appear in logs or error reports.
- Alerts exist for brute force, error spikes and admin changes.
- You know where to look, and you have a short runbook.

WATCH OUT FOR

- **Logging too much personal data** creates a privacy problem. Log IDs, not full profiles.
- **Serverless console logs vanish quickly** on some hosts. Send them to a log service with retention.

09

IDENTITY & ACCESS | CRITICAL | 20–40 MIN

09/27 SECURITY PASS

Lock down my admin routes

Hiding the Admin button isn't security. The server checks the role on every admin request.

WHAT THIS MEANS

Every admin page **and** every admin API behind it must check, on the server, that the current user really has the admin role. That role must be stored somewhere users can't edit. An unguessable URL, a hidden link or an email check in front-end code doesn't count.

WHAT GOES WRONG

Ben is a normal user. He types `/admin`, or simply calls `POST /api/admin/delete-user` that he spotted in the JavaScript bundle. It works, because only the UI was hiding the button. In the video, the fix is "role verified" on the server.

2-MINUTE SELF-CHECK

- Log in as a normal user and visit `/admin` directly. Expect a redirect or 403.
- As admin, copy an admin API request (Copy as cURL). Replace the cookie with a normal user's cookie and send it. Expect 403.
- Search your front-end bundle for admin email addresses or `isAdmin` logic.

> CLAUDE PROMPT · CHECK 09

Paste your Project Context first, then this

```
Act as a senior application security engineer. Make sure only real admins can reach admin functionality, enforced on the server.

AUDIT
1. List every admin, staff, internal or privileged route: pages (/admin, /dashboard/admin…), API routes, server actions, GraphQL mutations, database functions, cron and job endpoints, and any seed/test/debug/migration routes (/api/seed, /api/test, /api/reset-db…).
2. For each, state whether the role is checked on the SERVER inside the handler, for both the page and every data endpoint it calls.
3. Find where the role comes from. Acceptable: a roles column/table in my database, or server-only provider metadata (Supabase app_metadata, Clerk publicMetadata, Firebase custom claims). NOT acceptable: Supabase user_metadata (users can edit it), localStorage, a client-set cookie, a request parameter, or a hard-coded email check in front-end code.
4. Check whether admin email addresses or admin endpoint names are exposed in the client bundle.

FIX
- Create one server-side helper (e.g. requireAdmin()) that loads the verified user AND checks the role from a trusted source, returning 403 otherwise. Call it at the top of every admin handler, not only in middleware or layout files.
- Deny by default: new admin routes must be protected unless explicitly made public.
- Protect cron/webhook/internal endpoints with a long random secret in a header, compared in constant time, or the platform's built-in cron authentication.
- Remove seed/test/debug endpoints from production builds or block them by environment.
- Log every admin action (who, what, when, target) using Check 08's logger.
- Tell me how to require MFA for admin accounts with my auth provider.

VERIFY
Tests that prove: signed-out → 401; normal user → 403 on every admin page and endpoint; admin → allowed; a user can't make themselves admin by editing their own profile or metadata.

HOW TO WORK
- Audit first, and show me what you found before making large changes.
```

```
- Fix Critical and High issues directly. Ask me before any change users would notice (logging people
out, blocking a feature, changing a flow).
- Fix the root cause with the smallest safe change. Don't remove features, weaken other protections, or
leave "TODO: secure later" comments.
- If you add a package, name it, say why, and confirm it is a real, well-maintained package (not a look-
alike name).
- Never print real secret values; refer to them by variable name.
- If something lives outside the code (dashboards, hosting settings), give me exact click-by-click steps
instead.

FINISH WITH
1. Findings table: issue · file:line · severity · status (Fixed / Needs my action / Not applicable)
2. Files changed
3. How I can verify it myself in under 5 minutes
```

DONE WHEN

- Every admin page and admin API checks the role on the server.
- Roles live where users can't edit them.
- Normal users get 403 everywhere admin-related (tested).
- Seed/test/debug routes are gone from production.
- Admin actions are logged.

WATCH OUT FOR

- **Supabase `user_metadata` is user-editable.** Never store roles there. Use `app_metadata` or a table.
- **"Nobody knows the URL"** is not protection. URLs end up in bundles, logs and browser history.

10

DATA & INPUTS | CRITICAL | 15–30 MIN

10/27 SECURITY PASS

Use parameterized queries

The query's structure is fixed. User input is only ever treated as data.

WHAT THIS MEANS

SQL injection happens when user input is glued into a query string, so the input can change what the query does. Parameterized queries send the query and the values separately; whatever someone types is treated purely as data. ORMs do this for you, until you use their "raw" escape hatches.

WHAT GOES WRONG

Plantly's search builds `SELECT * FROM orders WHERE name = '${input}'`. Someone types `' OR '1'='1` and gets every order in the database. With other payloads, they can read the users table or delete data.

2-MINUTE SELF-CHECK

- Search your code for SQL built with template strings or `+`: `` `SELECT … ${ ``, `f"SELECT … {`, `"… WHERE id = " +`.
- Search for ORM escape hatches: `$queryRawUnsafe`, `$executeRawUnsafe`, `sequelize.literal`, `knex.raw(`, `.extra(`, `text(f"…")`.
- Type a single `'` into search boxes. A 500 error or SQL error message is a red flag.

> CLAUDE PROMPT · CHECK 10

Paste your Project Context first, then this

```
Act as a senior application security engineer. Eliminate injection vulnerabilities: SQL, NoSQL, filter-string and command injection.

AUDIT
1. List every database query in the codebase. Flag any query whose text is built from variables using string concatenation, template literals, f-strings, .format() or %.
2. Flag ORM escape hatches used with untrusted input: Prisma $queryRawUnsafe / $executeRawUnsafe / Prisma.raw (note: the tagged-template $queryRaw`...${value}` form IS parameterized and fine), Sequelize literal() and raw queries, Knex raw() with interpolation, TypeORM query() with strings, SQLAlchemy text() built with f-strings, Django raw()/extra(), Drizzle sql.raw().
3. Flag dynamic identifiers that can't be parameterized: ORDER BY column, sort direction, table or column names, LIMIT values built from input.
4. NoSQL (MongoDB etc.): flag places where request JSON goes straight into a query, which allows operator injection like {"$ne": null} or {"$gt": ""}.
5. Supabase/PostgREST: flag filter strings built from user input, for example .or(`name.eq.${input}`), and .rpc() calls whose SQL functions build dynamic SQL with EXECUTE.
6. Command injection: flag exec, spawn with shell: true, os.system, subprocess with shell=True, or any shell command that includes user input.

FIX
- Convert every flagged query to parameterized queries / bound parameters or safe ORM query builders.
- For dynamic identifiers, map user input to a hard-coded allowlist (e.g. sort=price → "price"); never insert the raw value.
- For NoSQL, validate input types (strings must be strings) before they reach queries.
- For shell commands, use argument arrays without a shell, or remove the need entirely.
- Confirm the database user has only the privileges the app needs.

VERIFY
Write tests that send ' OR '1'='1, '; DROP TABLE users; --, and {"$ne": null} to every affected input.
```

```
They must return normal "no results"/400 responses, never extra data or database errors.

HOW TO WORK
- Audit first, and show me what you found before making large changes.
- Fix Critical and High issues directly. Ask me before any change users would notice (logging people
out, blocking a feature, changing a flow).
- Fix the root cause with the smallest safe change. Don't remove features, weaken other protections, or
leave "TODO: secure later" comments.
- If you add a package, name it, say why, and confirm it is a real, well-maintained package (not a look-
alike name).
- Never print real secret values; refer to them by variable name.
- If something lives outside the code (dashboards, hosting settings), give me exact click-by-click steps
instead.

FINISH WITH
1. Findings table: issue · file:line · severity · status (Fixed / Needs my action / Not applicable)
2. Files changed
3. How I can verify it myself in under 5 minutes
```

DONE WHEN

- No query is built by string concatenation with user input.
- Raw ORM calls with user input are parameterized or removed.
- Sort/column inputs go through an allowlist.
- Injection test payloads return normal empty/400 responses.

WATCH OUT FOR

- **Prisma:** `$queryRaw`…${x}`` is safe; `$queryRawUnsafe(`…${x}`)` is not. The names look almost identical.
- **Database error messages** shown to users help attackers. Hide them (Check 20).

11

SECRETS & KEYS | CRITICAL | 15–25 MIN

11/27 SECURITY PASS

Check my environment variables

Every config value is in the right place: secrets stay server-side, and nothing secret is committed.

WHAT THIS MEANS

Environment variables keep configuration out of your code. But build tools ship any variable with a public prefix (`NEXT_PUBLIC_`, `VITE_`, `REACT_APP_`, `EXPO_PUBLIC_`) straight into the browser. You need a clean split between public and server-only values, a `.env` file that never reaches Git, and separate values for development and production.

WHAT GOES WRONG

In the video, `PUBLIC_PAY_SECRET` sits next to harmless public settings. The "public" prefix means the payment secret is bundled into every visitor's browser. Other classics: a committed `.env`, or a JWT secret that falls back to `"dev-secret"` in production.

2-MINUTE SELF-CHECK

- Run `git ls-files | grep -i env`. Only `.env.example` (with placeholders) should appear.
- Open your hosting dashboard's environment variables. Does any secret have a public prefix?
- Search your code for fallbacks like `|| "secret"` or `?? "changeme"`.

> CLAUDE PROMPT · CHECK 11

Paste your Project Context first, then this

```
Act as a senior application security engineer. Audit and clean up how this app uses environment
variables and configuration.

AUDIT
1. Find every environment variable the code reads (process.env, import.meta.env, os.environ, Deno.env,
config files) and every .env* file in the repo. Build a table: NAME · where it is used
(client/server/both) · public-safe or SECRET · notes. Show names only, never values.
2. Flag every SECRET with a public prefix (NEXT_PUBLIC_, VITE_, REACT_APP_, EXPO_PUBLIC_, PUBLIC_,
NUXT_PUBLIC_) or read inside client-side code.
3. Check .gitignore covers .env, .env.local, .env.*.local and similar. Check whether any .env file is
tracked by Git now or was in the past.
4. Flag insecure fallbacks, for example process.env.JWT_SECRET || "secret", or defaults that silently
use test values in production.
5. Check secrets are not exposed through other routes: /api/config or /api/health endpoints, error
messages, client-side logs, server-rendered page props, Docker images/build args, or CI logs.
6. Check development and production use separate values (Stripe test keys in dev and live keys only in
production, a separate database for dev).

FIX
- Rename or move secrets so none are public, and move secret-using code to the server.
- Add .env.example with every variable name and a safe placeholder, and update .gitignore.
- Add start-up validation (e.g. a Zod schema, t3-env or pydantic-settings) so the app fails fast with a
clear message if a required variable is missing, without printing its value.
- Remove insecure fallbacks. Make sure signing secrets are long random values (at least 32 bytes) and
tell me the command to generate one.

REPORT
List every variable that may have been exposed (public prefix, committed file, logs) so I can rotate it
in Check 25.
```

```
HOW TO WORK
- Audit first, and show me what you found before making large changes.
- Fix Critical and High issues directly. Ask me before any change users would notice (logging people
out, blocking a feature, changing a flow).
- Fix the root cause with the smallest safe change. Don't remove features, weaken other protections, or
leave "TODO: secure later" comments.
- If you add a package, name it, say why, and confirm it is a real, well-maintained package (not a look-
alike name).
- Never print real secret values; refer to them by variable name.
- If something lives outside the code (dashboards, hosting settings), give me exact click-by-click steps
instead.

FINISH WITH
1. Findings table: issue · file:line · severity · status (Fixed / Needs my action / Not applicable)
2. Files changed
3. How I can verify it myself in under 5 minutes
```

DONE WHEN

- No secret has a public prefix or is read by client code.
- `.env` files are ignored by Git; `.env.example` documents every variable.
- The app fails fast when a required variable is missing.
- Dev and production use different keys and databases.

WATCH OUT FOR

- **Adding `.env` to `.gitignore` doesn't remove it from Git history.** That's Check 24.
- **Changing an env var** on most hosts requires a redeploy to take effect.

12

NETWORK & BROWSER | HIGH | 20–40 MIN

12/27 SECURITY PASS

Add rate limiting

Nobody can hammer your login, forms or paid APIs thousands of times a minute.

WHAT THIS MEANS

Rate limiting caps how many requests one person, IP or account can make in a time window. It stops password guessing, credential stuffing, spam sign-ups, scraping and, very important for AI apps, bots running up your bill on expensive endpoints.

WHAT GOES WRONG

A bot tries 10,000 leaked passwords against Plantly's `/api/login`. Or someone writes a 5-line loop against `/api/generate`, which calls an AI model, and you wake up to a huge invoice. In the video, the fix is a 429 "slow down" response.

2-MINUTE SELF-CHECK

- Against **your own** app, send 30 quick login attempts:

  ```
  for i in $(seq 1 30); do curl -s -o /dev/null -w "%{http_code} " -X POST https://YOURAPP/api/login -H 'content-type: application/json' -d '{}'; done
  ```

  You should start seeing `429`.
- Try the same against your AI, sign-up and contact-form endpoints.

> CLAUDE PROMPT · CHECK 12

Paste your Project Context first, then this

```
Act as a senior application security engineer. Add rate limiting that protects this app from brute force, spam and cost abuse.

AUDIT
1. List the endpoints that need limits, grouped as: authentication (login, sign-up, password reset, OTP/MFA verification, magic links), expensive or paid (AI/LLM calls, email/SMS sending, file processing, search, exports), user-generated content (comments, contact forms), and the general API.
2. Check what limiting exists today, and whether it would actually work on my hosting. In-memory counters do NOT work on serverless or with multiple instances.

FIX
1. Propose a limits table for my review, starting from these defaults:
   - Login: 5 attempts per minute per IP+email, with increasing delays after repeated failures
   - Sign-up: 5 per hour per IP
   - Password reset / magic link: 3 per hour per email and 10 per hour per IP
   - OTP/MFA code: 5 attempts per code, then the code is invalidated
   - AI/LLM endpoints: per-user per-minute limits PLUS a daily quota per user/plan
   - Email/SMS sending: strict per-user and global caps
   - General API: a generous per-IP limit (e.g. 100 per minute)
2. Implement it with a shared store that works on my hosting (e.g. Upstash Redis with @upstash/ratelimit, Redis, the database, or platform features like Vercel WAF or Cloudflare rate limiting). Key by user ID when signed in, and by IP otherwise.
3. Get the client IP correctly for my host. Don't trust a raw X-Forwarded-For value that clients can spoof; use the platform's verified header.
4. Return 429 with a Retry-After header and a friendly message. Responses must not reveal whether an account exists.
5. Log rate-limit hits (Check 08).
6. For paid APIs, also tell me how to set hard spending limits in each provider's dashboard as a last line of defence.
```

```
VERIFY
Give me a script that fires requests at each limited endpoint on localhost or staging and shows the
switch to 429.

HOW TO WORK
- Audit first, and show me what you found before making large changes.
- Fix Critical and High issues directly. Ask me before any change users would notice (logging people
out, blocking a feature, changing a flow).
- Fix the root cause with the smallest safe change. Don't remove features, weaken other protections, or
leave "TODO: secure later" comments.
- If you add a package, name it, say why, and confirm it is a real, well-maintained package (not a look-
alike name).
- Never print real secret values; refer to them by variable name.
- If something lives outside the code (dashboards, hosting settings), give me exact click-by-click steps
instead.

FINISH WITH
1. Findings table: issue · file:line · severity · status (Fixed / Needs my action / Not applicable)
2. Files changed
3. How I can verify it myself in under 5 minutes
```

DONE WHEN

- Login, sign-up, reset, OTP and paid endpoints are rate-limited.
- Limits use a shared store that works on your hosting.
- Over-limit requests get a 429 with Retry-After.
- Provider-side spending caps are set on AI/email/SMS services.

WATCH OUT FOR

- **SMS endpoints are a fraud magnet** ("SMS pumping"). Cap them hard and restrict destination countries.
- **AI endpoints without quotas** are the #1 surprise bill for vibe-coded apps.

13

NETWORK & BROWSER | MEDIUM | 10–20 MIN

13/27 SECURITY PASS

Restrict my CORS settings

Only your own websites can read your API's responses from a browser.

WHAT THIS MEANS

CORS tells browsers which *other* websites may read responses from your API using a visitor's browser. A wildcard or "reflect any origin" setting, especially together with cookies, lets a malicious site quietly read a logged-in user's data. CORS is not authentication; it only controls browsers, so your API still needs Checks 01 and 09.

WHAT GOES WRONG

Plantly's API replies to any origin with `Access-Control-Allow-Origin` set to that origin, plus `Allow-Credentials: true`. A user visits `other-site.example`, whose script fetches `plantly.app/api/me` with the user's cookies and reads their data. In the video, the fix is "blocked by the browser".

2-MINUTE SELF-CHECK

- `curl -s -I -H "Origin: https://evil.example" https://YOURAPP/api/ANY-ENDPOINT`
- If the response echoes `Access-Control-Allow-Origin: https://evil.example`, or shows `*` together with `Allow-Credentials: true`, fix it.

> CLAUDE PROMPT · CHECK 13

Paste your Project Context first, then this

```
Act as a senior application security engineer. Lock down this app's CORS configuration.

AUDIT
1. Find every place CORS is configured: cors() middleware, framework config (next.config.js headers,
vercel.json, netlify.toml/_headers), individual API routes or edge/serverless functions that set Access-
Control-* headers manually, Supabase Edge Functions, API gateways, and cloud storage bucket CORS (S3,
R2, GCS, Firebase Storage).
2. For each, report: allowed origins, whether credentials are allowed, allowed methods and headers.
3. Flag: Access-Control-Allow-Origin: *, origins reflected from the request without checking, "null"
allowed as an origin, wildcard combined with credentials, and loose matching (endsWith("plantly.app")
also matches evilplantly.app; a regex without anchors).
4. Tell me whether the API is even called cross-origin. If my front end and API share an origin, I may
not need CORS at all.

FIX
- Replace wildcards and reflection with an explicit allowlist read from an environment variable
(production domains; localhost only in development). Use exact string matches.
- Allow credentials only if truly needed, and never together with a wildcard.
- Restrict methods and headers to what the front end actually uses.
- Public, read-only, non-personalised endpoints (like a public product feed) may use * but must never
use cookies.
- Explain in one paragraph why CORS is not authentication and confirm the endpoints still enforce auth
on the server.

VERIFY
Give me curl commands showing an allowed origin receives the CORS headers and an unknown origin does
not.

HOW TO WORK
- Audit first, and show me what you found before making large changes.
- Fix Critical and High issues directly. Ask me before any change users would notice (logging people
```

```
out, blocking a feature, changing a flow).
- Fix the root cause with the smallest safe change. Don't remove features, weaken other protections, or
leave "TODO: secure later" comments.
- If you add a package, name it, say why, and confirm it is a real, well-maintained package (not a look-
alike name).
- Never print real secret values; refer to them by variable name.
- If something lives outside the code (dashboards, hosting settings), give me exact click-by-click steps
instead.

FINISH WITH
1. Findings table: issue · file:line · severity · status (Fixed / Needs my action / Not applicable)
2. Files changed
3. How I can verify it myself in under 5 minutes
```

DONE WHEN

- No wildcard or reflected origin on endpoints that use cookies or return private data.
- Origins are an exact allowlist from config.
- An unknown origin gets no `Access-Control-Allow-Origin` header.

WATCH OUT FOR

- **CORS only affects browsers.** curl, scripts and servers ignore it completely, so it never replaces auth.
- **Storage buckets have their own CORS** settings, separate from your app.

14

NETWORK & BROWSER | MEDIUM | 20–45 MIN

14/27 SECURITY PASS

Add security headers

A few HTTP headers switch on powerful browser protections for free.

WHAT THIS MEANS

Security headers tell the browser how to treat your site: which scripts it may run (CSP), never to use HTTP (HSTS), never to be framed by other sites (anti-clickjacking), not to guess file types, and what to share in the Referer. They don't fix bugs, but they make many attacks fail or do far less damage.

WHAT GOES WRONG

Without frame protection, `other-site.example` loads Plantly in an invisible iframe and tricks users into clicking "Delete account". Without a CSP, any XSS bug can load scripts from anywhere. In the video, `X-Frame-Options: DENY` blocks the framing.

2-MINUTE SELF-CHECK

- Scan your domain at **securityheaders.com**. Aim for A.
- Or run `curl -s -I https://YOURAPP` and look for `content-security-policy`, `strict-transport-security`, `x-content-type-options`, `x-frame-options`, `referrer-policy`.

> CLAUDE PROMPT · CHECK 14

Paste your Project Context first, then this

```
Act as a senior application security engineer. Add a strong, working set of HTTP security headers to every response from this app.

AUDIT
1. Show me which security headers are currently sent, and where headers are configured (framework config, middleware, helmet, vercel.json, netlify _headers, nginx/Caddy).
2. List every third-party origin the app legitimately loads from or connects to (analytics, fonts, Stripe, Supabase, auth provider, image CDN, AI API, maps) by checking the code. These are needed for the CSP.

FIX
Add these headers for all routes, in the single most appropriate place for my stack:
- Content-Security-Policy, starting from: default-src 'self'; script-src 'self'; style-src 'self'; img-src 'self' data: https:; font-src 'self'; connect-src 'self'; object-src 'none'; base-uri 'self'; form-action 'self'; frame-ancestors 'none'; upgrade-insecure-requests. Add only the specific third-party origins found above. Avoid 'unsafe-inline' and 'unsafe-eval' for scripts; if the framework needs inline scripts, use nonces or hashes. If something truly requires an unsafe value, explain why.
- Strict-Transport-Security: max-age=63072000; includeSubDomains
- X-Content-Type-Options: nosniff
- X-Frame-Options: DENY (or SAMEORIGIN if I must frame my own pages)
- Referrer-Policy: strict-origin-when-cross-origin
- Permissions-Policy: camera=(), microphone=(), geolocation=(). Enable only features the app uses.
- Cross-Origin-Opener-Policy: same-origin (use same-origin-allow-popups if OAuth or payment popups break)
- Remove X-Powered-By and server version headers.
- Cache-Control: no-store on authenticated pages and API responses containing personal data.

ROLLOUT
Ship the CSP first as Content-Security-Policy-Report-Only, list what would be blocked on each main page (login, checkout, OAuth, payments, uploads, embeds), adjust, and then switch to enforcing. Tell me exactly which pages to click through to confirm nothing broke.
```

```
VERIFY
Give me the curl command to confirm every header, and tell me what grade to expect on
securityheaders.com.

HOW TO WORK
- Audit first, and show me what you found before making large changes.
- Fix Critical and High issues directly. Ask me before any change users would notice (logging people
out, blocking a feature, changing a flow).
- Fix the root cause with the smallest safe change. Don't remove features, weaken other protections, or
leave "TODO: secure later" comments.
- If you add a package, name it, say why, and confirm it is a real, well-maintained package (not a look-
alike name).
- Never print real secret values; refer to them by variable name.
- If something lives outside the code (dashboards, hosting settings), give me exact click-by-click steps
instead.

FINISH WITH
1. Findings table: issue · file:line · severity · status (Fixed / Needs my action / Not applicable)
2. Files changed
3. How I can verify it myself in under 5 minutes
```

DONE WHEN

- All listed headers are present on every page and API response.
- The CSP is enforced (not just report-only) and the app still works: login, checkout, OAuth.
- `X-Powered-By` is gone.
- securityheaders.com grade is A or better.

WATCH OUT FOR

- **A broken CSP silently breaks features** like payments, OAuth popups and analytics. That's why report-only comes first.
- **Headers set in `next.config.js`** don't apply to every host setup. Verify on the live URL, not just locally.

15

IDENTITY & ACCESS | HIGH | 15–25 MIN

15/27 SECURITY PASS

Secure my session cookies

The cookie that keeps users logged in can't be read by scripts, sent over HTTP, or abused by other sites.

WHAT THIS MEANS

Whoever holds the session cookie **is** the user. Three flags protect it: **HttpOnly** (JavaScript can't read it, so XSS can't steal it), **Secure** (only sent over HTTPS) and **SameSite=Lax** (not sent on most cross-site requests). Sessions should also expire, be renewed at login, and be destroyed on logout.

WHAT GOES WRONG

Plantly stores its auth token in `localStorage`. One XSS bug and an attacker's script reads it and sends it home, getting a fully logged-in copy of the user's account that lasts for 30 days. In the video, the cookie is protected by Secure, HttpOnly and SameSite=Lax.

2-MINUTE SELF-CHECK

- DevTools → Application → Cookies. Your session cookie should show ✓ HttpOnly, ✓ Secure and SameSite Lax or Strict.
- DevTools → Application → Local Storage. Look for anything named `token`, `jwt`, `session` or `access_token`.
- Log out, then replay an old request with the old cookie (Copy as cURL). It should fail.

> CLAUDE PROMPT · CHECK 15

Paste your Project Context first, then this

```
Act as a senior application security engineer. Harden how this app stores and manages user sessions.

AUDIT
1. Find where session state lives: server-side session store, cookies (which library sets them), JWTs,
localStorage/sessionStorage, or a provider SDK. List every cookie the app sets with its current flags.
2. Check each auth/session cookie for: HttpOnly, Secure, SameSite (Lax or Strict), Path=/, no
unnecessarily broad Domain attribute, and sensible Max-Age/Expires.
3. Check session lifecycle: is the session ID regenerated on login and on privilege change (prevents
session fixation)? Is it destroyed server-side on logout? Do sessions have an idle timeout and an
absolute maximum lifetime? Are all sessions revoked after a password change or reset?
4. If tokens are kept in localStorage or sessionStorage, explain the risk, and whether the provider
supports cookie-based sessions instead.
5. If JWTs are used: check lifetime (short-lived access tokens with refresh-token rotation is the
standard), how revocation works, and that no sensitive data is stored in the payload (JWT payloads are
readable by anyone).

FIX
- Set HttpOnly, Secure and SameSite=Lax on all auth cookies (Strict for extra-sensitive apps if it
doesn't break login flows). Use the __Host- prefix where the framework allows it.
- Regenerate sessions on login, destroy them on logout, and add a "log out of all devices" action that
runs automatically after a password change.
- Pick sensible lifetimes, for example 7–30 days with an idle timeout for a typical consumer app;
shorter for admin sessions. Explain your choice.
- If my auth provider manages cookies itself, confirm its defaults are secure and that my code doesn't
override them insecurely.
- Never put secrets or personal data in readable cookies; sign or encrypt any cookie that must hold
data, using a strong secret.
```

```
VERIFY
List the final cookies with their flags, and give me a test proving a logged-out cookie no longer works.

HOW TO WORK
- Audit first, and show me what you found before making large changes.
- Fix Critical and High issues directly. Ask me before any change users would notice (logging people
out, blocking a feature, changing a flow).
- Fix the root cause with the smallest safe change. Don't remove features, weaken other protections, or
leave "TODO: secure later" comments.
- If you add a package, name it, say why, and confirm it is a real, well-maintained package (not a look-
alike name).
- Never print real secret values; refer to them by variable name.
- If something lives outside the code (dashboards, hosting settings), give me exact click-by-click steps
instead.

FINISH WITH
1. Findings table: issue · file:line · severity · status (Fixed / Needs my action / Not applicable)
2. Files changed
3. How I can verify it myself in under 5 minutes
```

DONE WHEN

- Session cookies have HttpOnly, Secure and SameSite set.
- No long-lived tokens in localStorage, or the risk is understood and mitigated.
- Logout and password change invalidate sessions server-side.
- Sessions expire on a sensible schedule.

WATCH OUT FOR

- **Some auth SDKs store tokens in localStorage by default.** That can be acceptable with a strong CSP, but know which one you're using and why.
- **SameSite=Strict** can break login redirects from emails or OAuth. Lax is the usual default.

16

NETWORK & BROWSER | HIGH | 15–30 MIN

16/27 SECURITY PASS

Add CSRF protection

Other websites can't make your logged-in users perform actions without their knowledge.

WHAT THIS MEANS

Cross-site request forgery (CSRF) abuses the fact that browsers automatically attach your cookies to requests. A malicious page can submit a hidden form to your app, and if you only check the cookie, your server thinks the user did it. Protection: SameSite cookies, a CSRF token or Origin check on every state-changing request, and never changing data via GET.

WHAT GOES WRONG

In the video, `other-site.example` hides a form that posts to Plantly's "Change email" endpoint. Ana visits the page while logged in, her browser sends the request with her cookie, and her email is changed to the attacker's, who then resets her password. The fix: a token check, with the forged request rejected.

2-MINUTE SELF-CHECK

- Does your app use cookies for login? If yes, CSRF applies to you.
- Look for any GET route that changes data (`/delete?id=` , `/logout` , `/subscribe`).
- On your own app: save a local HTML file with an auto-submitting form that posts to one of your endpoints, open it while logged in, and see whether the action happens.

> CLAUDE PROMPT · CHECK 16

Paste your Project Context first, then this

```
Act as a senior application security engineer. Protect this app against cross-site request forgery (CSRF).

AUDIT
1. Determine how requests are authenticated. Cookies (including provider-managed session cookies) mean CSRF applies. If auth is ONLY a bearer token in an Authorization header added by JavaScript, CSRF risk is low; say so, and confirm no cookie-based path exists.
2. List every state-changing endpoint (POST, PUT, PATCH, DELETE, server actions, form handlers) and every GET route that changes state.
3. Check what protection exists: framework CSRF middleware (Django, Rails, Laravel have it built in), Next.js Server Actions' Origin/Host check (and whether serverActions.allowedOrigins is loosened), SameSite cookie settings, custom token or Origin checks.
4. Check whether JSON endpoints also accept form-encoded or text/plain bodies. Those can be sent cross-site without a CORS preflight.

FIX
- Move any state-changing GET to POST/DELETE.
- Use the framework's built-in CSRF protection where it exists, and make sure it isn't disabled. For Express or custom servers use a maintained library implementing synchronizer or signed double-submit tokens (the old "csurf" package is deprecated).
- Add an Origin header check for every state-changing request as defence in depth: reject when Origin is present and isn't my own site.
- Make JSON endpoints require Content-Type: application/json.
- Keep auth cookies SameSite=Lax or Strict (Check 15).
- Exempt only webhooks, which must instead verify their signatures (Check 19).
- For high-risk actions (change email, change password, delete account, payouts), also require the current password or a recent re-login.

VERIFY
```

```
Write a test that sends a state-changing request with a foreign Origin and no CSRF token and confirms it is rejected, and that the real UI still works.

HOW TO WORK
- Audit first, and show me what you found before making large changes.
- Fix Critical and High issues directly. Ask me before any change users would notice (logging people out, blocking a feature, changing a flow).
- Fix the root cause with the smallest safe change. Don't remove features, weaken other protections, or leave "TODO: secure later" comments.
- If you add a package, name it, say why, and confirm it is a real, well-maintained package (not a look-alike name).
- Never print real secret values; refer to them by variable name.
- If something lives outside the code (dashboards, hosting settings), give me exact click-by-click steps instead.

FINISH WITH
1. Findings table: issue · file:line · severity · status (Fixed / Needs my action / Not applicable)
2. Files changed
3. How I can verify it myself in under 5 minutes
```

DONE WHEN

- No GET request changes data.
- All state-changing requests require a CSRF token or pass an Origin check.
- Cookies use SameSite=Lax or Strict.
- Sensitive account changes require re-authentication.

WATCH OUT FOR

- **SameSite helps but isn't complete:** subdomains you don't control and older browsers can still be problems.
- **Login CSRF** (logging a victim into the attacker's account) is real too. Protect the login form.

17

IDENTITY & ACCESS | HIGH | 15–30 MIN

17/27 SECURITY PASS

Make password-reset links expire

Reset links are short-lived, single-use, random, and tied to one account.

WHAT THIS MEANS

A password-reset link is a temporary master key to an account. It must be unguessable, work only once, expire quickly (about 15–30 minutes), and stop working when a newer link is requested or the password changes. The same rules apply to magic-login links, email-verification links and invites.

WHAT GOES WRONG

An old reset email from three months ago sits in a user's inbox. When their email account is breached, the link still works and the attacker takes over their Plantly account. Or tokens are predictable (like a timestamp or user ID), so attackers just generate valid links. In the video, the old link is expired and a used link can't be reused.

2-MINUTE SELF-CHECK

- Request a reset and use the link. Click it again: it must fail.
- Request two resets. The first link must stop working.
- Request a reset for an email that doesn't exist. The message must be identical to a real one.
- Wait past the expiry time and try again.

> CLAUDE PROMPT · CHECK 17

Paste your Project Context first, then this

```
Act as a senior application security engineer. Make the password-reset flow, and every similar emailed link, secure.

SCOPE
Password reset, magic-link login, email verification, email-change confirmation, team invites, and any other link or code sent by email or SMS that grants access.

AUDIT
For each flow, report: how the token is generated, how it is stored, how long it lasts, whether it's single-use, what invalidates it, what the user sees for unknown emails, and how the link URL is built.

FIX (if I use an auth provider such as Supabase, Clerk or Firebase, show me which provider settings to check, and only change code I own)
1. Generate tokens from a cryptographically secure random source with at least 32 bytes (crypto.randomBytes(32) / secrets.token_urlsafe(32)). No timestamps, user IDs, UUIDv1 or Math.random.
2. Store only a SHA-256 hash of the token, never the token itself.
3. Expiry: 15–30 minutes for password reset and magic links, up to 24 hours for email verification, up to 7 days for invites.
4. Single use: invalidate on use. Also invalidate all outstanding tokens for that user when a new one is requested or the password changes.
5. Show the same message and similar response time whether or not the email exists ("If an account exists, we've sent a link").
6. Rate limit requests per email and per IP (Check 12).
7. Build the link from a configured APP_URL environment variable, NEVER from the request's Host header. Host-header poisoning can send reset links to an attacker's domain.
8. On the reset page: set Referrer-Policy: no-referrer and load no third-party scripts (analytics) that could see the token in the URL.
9. After a successful reset: apply the normal password policy and hashing (Check 05), revoke all existing sessions, and email the user a "your password was changed" notice.
```

```
10. Allowlist redirect URLs used after the flow completes.

VERIFY
Tests proving: a used token fails; an expired token fails; an older token fails after a newer one is
issued; responses for real and fake emails are identical; and a forged Host header does not change the
link domain.

HOW TO WORK
- Audit first, and show me what you found before making large changes.
- Fix Critical and High issues directly. Ask me before any change users would notice (logging people
out, blocking a feature, changing a flow).
- Fix the root cause with the smallest safe change. Don't remove features, weaken other protections, or
leave "TODO: secure later" comments.
- If you add a package, name it, say why, and confirm it is a real, well-maintained package (not a look-
alike name).
- Never print real secret values; refer to them by variable name.
- If something lives outside the code (dashboards, hosting settings), give me exact click-by-click steps
instead.

FINISH WITH
1. Findings table: issue · file:line · severity · status (Fixed / Needs my action / Not applicable)
2. Files changed
3. How I can verify it myself in under 5 minutes
```

DONE WHEN

- Tokens are random (32+ bytes), stored hashed, and expire in 15–30 minutes.
- Links are single-use; requesting a new one kills the old one.
- Responses don’t reveal whether an email is registered.
- Links are built from a configured URL, not the Host header.
- A password reset logs out all other sessions and notifies the user.

WATCH OUT FOR

- **Analytics scripts on the reset page** can record the full URL, token included.
- **Provider settings matter:** Supabase, Firebase and Clerk each have their own expiry and redirect-URL settings. Check them.

18

DATA & INPUTS | HIGH | 20–45 MIN

18/27 SECURITY PASS

Restrict file upload types and sizes

Only expected files, of reasonable size, stored and served safely.

WHAT THIS MEANS

Uploads are one of the riskiest features you can build. Files can be malware, scripts disguised as images, HTML or SVG pages that run code on your domain, gigantic files that eat your storage and bill, or filenames that try to write outside the upload folder. The server must check the type, size and actual content, and store files where they can't run.

WHAT GOES WRONG

Plantly allows avatar uploads. Someone uploads `invoice.exe`, or an SVG containing JavaScript. It's served from `plantly.app/uploads/`, so the script runs with full access to logged-in users' sessions. In the video, the upload check verifies file type, size and content: the image is accepted and the program is rejected.

2-MINUTE SELF-CHECK

- Rename an `.html` file to `.jpg` and upload it. Open the resulting URL: does it render as a web page?
- Upload a file much larger than the stated limit.
- Upload an SVG containing `<script>alert(1)</script>` and open it directly.

> CLAUDE PROMPT · CHECK 18

Paste your Project Context first, then this

```
Act as a senior application security engineer. Make every file upload in this app safe.

AUDIT
1. Find every upload path: forms, avatars, attachments, CSV/Excel imports, drag-and-drop,
presigned/direct-to-storage uploads (S3, R2, Supabase Storage, Firebase Storage, UploadThing,
Cloudinary), and AI features that accept files.
2. For each, report: who can upload (auth required?), what checks exist and where (client only or
server), where files are stored, how they're named, how they're served (domain, Content-Type, Content-
Disposition), and whether they are public.

FIX (all checks enforced on the server or in storage policies, never only in the browser)
1. Type: an allowlist per feature (for example avatars: jpeg, png, webp). Check the extension AND the
real content via magic bytes (e.g. the file-type library); don't trust the client's Content-Type. Block
HTML, SVG, XML, JS and executables unless a feature truly needs them.
2. Size: a maximum per file (for example 5 MB for images) enforced server-side and in presigned URL
conditions (content-length-range) or storage bucket limits; plus a maximum number of files per request
and a per-user storage quota.
3. Names and paths: generate random names (UUIDs). Never use the user's filename in a storage path
(prevents ../ path traversal). Keep the original name only as escaped display text.
4. Storage: object storage or a folder outside the web root, private by default. Serve private files
through short-lived signed URLs after an authorization check.
5. Serving: correct Content-Type, X-Content-Type-Options: nosniff, and Content-Disposition: attachment
for anything that isn't an image you display. Ideally serve user files from a separate domain.
6. Images: re-encode with sharp or Pillow (this strips hidden payloads and EXIF data, including GPS
location) and cap pixel dimensions to stop decompression bombs.
7. Archives and documents: cap uncompressed size and file count for zips. For CSV exports, prefix cells
starting with =, +, - or @ to prevent spreadsheet formula injection.
8. If files are shared between users, scan them for malware or tell me which service to use.

VERIFY
Tests that upload a renamed .html, a script-bearing .svg, an oversized file and a ../ filename. All must
```

```
be rejected or neutralised.

HOW TO WORK
- Audit first, and show me what you found before making large changes.
- Fix Critical and High issues directly. Ask me before any change users would notice (logging people
out, blocking a feature, changing a flow).
- Fix the root cause with the smallest safe change. Don't remove features, weaken other protections, or
leave "TODO: secure later" comments.
- If you add a package, name it, say why, and confirm it is a real, well-maintained package (not a look-
alike name).
- Never print real secret values; refer to them by variable name.
- If something lives outside the code (dashboards, hosting settings), give me exact click-by-click steps
instead.

FINISH WITH
1. Findings table: issue · file:line · severity · status (Fixed / Needs my action / Not applicable)
2. Files changed
3. How I can verify it myself in under 5 minutes
```

DONE WHEN

- Allowed types are checked by real file content, not just the extension.
- Size limits are enforced on the server or in storage.
- Files get random names and can’t escape their folder.
- Uploaded files can’t execute as pages on your main domain.
- Images are re-encoded and location data stripped.

WATCH OUT FOR

- **Presigned uploads skip your server**, so limits must live in the presigned policy or bucket rules.
- **Public buckets** are often listable. Make sure nobody can browse everyone’s files.

19

DATA & INPUTS | CRITICAL | 30–60 MIN

19/27 SECURITY PASS

Verify prices and payments

The server decides what things cost, and only a verified payment unlocks anything.

WHAT THIS MEANS

The browser is the customer's territory; they can edit anything in it. So the server must compute every price, discount and total from its own database, and must only fulfil an order or unlock a plan when the payment provider confirms it through a **signature-verified webhook**, never because someone landed on a "success" page.

WHAT GOES WRONG

In the video, the Ceramic planter costs $49.00, but the checkout request is edited to say $1.00 and the server must catch the price mismatch. Another favourite: visiting `/checkout/success` directly marks the order as paid, or a fake webhook call upgrades an account to Pro for free.

2-MINUTE SELF-CHECK

- In DevTools, edit the price, amount or quantity in your checkout request and send it. The charged amount must not change.
- Visit your success/thank-you URL directly without paying. Nothing should be unlocked.
- Try applying the same coupon twice, an expired coupon, and a quantity of `0` or `-1`.

> CLAUDE PROMPT · CHECK 19

Paste your Project Context first, then this

```
Act as a senior application security engineer specialising in payments. Make sure nobody can pay less
than they should, or get something without paying.

AUDIT
Trace the whole money flow end to end: product/plan pages → cart → checkout → payment provider (Stripe,
Lemon Squeezy, Paddle, PayPal, Razorpay, RevenueCat…) → success page → webhook → fulfilment/entitlement
→ refunds/cancellations. For each step, say what the client sends and what the server trusts.

FIX
1. Prices: the client sends only product/plan IDs and quantities. The server looks up prices from the
database or provider price IDs and calculates every total, tax, shipping and discount itself. Ignore any
price, amount, currency or total sent by the client.
2. Quantities: integers with a minimum of 1 and a sensible maximum; reject everything else.
3. Money as integers in minor units (cents/paise), never floating point.
4. Coupons and credits: validate on the server (exists, not expired, usage limits, per-user limits,
eligible products) and apply them inside a database transaction so they can't be used twice through
parallel requests.
5. Fulfilment ONLY from verified webhooks (e.g. Stripe checkout.session.completed /
payment_intent.succeeded / invoice.paid), never from the success redirect. The success page just shows
status.
6. Webhooks: verify the signature using the RAW request body and the endpoint secret (e.g.
stripe.webhooks.constructEvent); reject anything that fails. Store processed event IDs so duplicates are
ignored (idempotency), handle events arriving out of order, and check that the amount and currency paid
match the order.
7. Subscriptions and entitlements: decide access on the server from subscription status stored via
webhooks, never from a client flag or a query parameter like ?plan=pro. Handle cancellations, failed
payments, refunds and disputes so access is removed when it should be.
8. Balances, credits and stock: update atomically in transactions to prevent race-condition double
spending.
9. Card data never touches my server; use the provider's hosted checkout or elements.
10. Test keys in development, live keys only in production; the webhook route is exempt from CSRF but
```

```
signature-verified.

VERIFY
Write tests for: a tampered price, a negative quantity, a reused coupon, visiting the success URL
without paying, an unsigned/forged webhook, and the same webhook delivered twice. Then give me the
provider's CLI or dashboard steps to send test webhooks (e.g. stripe listen / stripe trigger).

HOW TO WORK
- Audit first, and show me what you found before making large changes.
- Fix Critical and High issues directly. Ask me before any change users would notice (logging people
out, blocking a feature, changing a flow).
- Fix the root cause with the smallest safe change. Don't remove features, weaken other protections, or
leave "TODO: secure later" comments.
- If you add a package, name it, say why, and confirm it is a real, well-maintained package (not a look-
alike name).
- Never print real secret values; refer to them by variable name.
- If something lives outside the code (dashboards, hosting settings), give me exact click-by-click steps
instead.

FINISH WITH
1. Findings table: issue · file:line · severity · status (Fixed / Needs my action / Not applicable)
2. Files changed
3. How I can verify it myself in under 5 minutes
```

DONE WHEN

- Totals are calculated only on the server from trusted prices.
- Orders and plans unlock only from signature-verified webhooks.
- Duplicate webhooks don’t double-fulfil.
- Coupons, credits and stock can’t be double-spent.
- Refunds and cancellations remove access.

WATCH OUT FOR

- **Parsing the webhook body as JSON before verifying** breaks signature checks. Many frameworks need raw-body config on that route.
- **“Free trial” and referral logic** are business-logic money leaks too. Make sure they can’t be reused with new emails endlessly.

20

DEPLOYMENT & SUPPLY CHAIN | HIGH | 10–20 MIN

20/27 SECURITY PASS

Disable production debug mode

Errors tell users “something went wrong”, and tell you everything else, privately.

WHAT THIS MEANS

Debug mode shows detailed errors: stack traces, file paths, SQL queries, environment values, sometimes even an interactive console. That’s gold for attackers. In production, users should see a generic message with a reference ID, while full details go to your logs.

WHAT GOES WRONG

In the video, a debug page reveals the database host, a secret key, the server’s file path and the SQL query, a complete map for an attacker. Some debug consoles (like Flask/Werkzeug’s) go further and let anyone run code on your server.

2-MINUTE SELF-CHECK

- On the live site, visit a page that doesn’t exist, send malformed JSON to an API, and use an invalid ID. You should see clean errors, never stack traces.
- Check whether `.map` files are downloadable from your live site (look in DevTools → Sources).
- Try `/docs`, `/redoc`, `/graphql` and `/swagger` on your live domain.

> CLAUDE PROMPT · CHECK 20

Paste your Project Context first, then this

```
Act as a senior application security engineer. Make sure production never exposes debug information or developer tooling.

AUDIT
1. Identify the framework(s) and check their production debug settings, for example:
   - Django: DEBUG = False and ALLOWED_HOSTS set. Flask: debug off (the Werkzeug debugger allows remote code execution). FastAPI: debug=False, and /docs, /redoc and /openapi.json disabled or protected if the API is private.
   - Node/Express: NODE_ENV=production and an error handler that never sends err.stack or err.message from internal errors.
   - Next.js/Vite/React: a production build is deployed (not a dev server); no detailed errors in API responses.
   - Laravel: APP_DEBUG=false. Rails: consider_all_requests_local = false. Spring: Actuator endpoints restricted.
   - GraphQL: introspection and playground/GraphiQL disabled in production unless intentionally public.
2. Check every error path: API catch blocks that return error.message or the raw database error, 404/500 pages, validation errors.
3. Check public source maps, directory listing, verbose headers (X-Powered-By, server versions), debug/test routes, health or status endpoints that reveal versions, environment or dependencies, and client-side console.log calls that print sensitive data.

FIX
- Turn off debug mode in production through environment config, so it can't be forgotten.
- Add a global error handler: users get a generic message plus a request ID; full details go only to server logs/error tracking (Check 08).
- Map database and third-party errors to generic messages.
- Don't serve source maps publicly (or upload them privately to the error tracker instead).
- Disable directory listing and remove debug-only routes and noisy console logs from production.

VERIFY
Give me curl commands that trigger a 404, a 500 and a validation error on production and show that none reveal stack traces, paths, queries or versions.
```

```
HOW TO WORK
- Audit first, and show me what you found before making large changes.
- Fix Critical and High issues directly. Ask me before any change users would notice (logging people
out, blocking a feature, changing a flow).
- Fix the root cause with the smallest safe change. Don't remove features, weaken other protections, or
leave "TODO: secure later" comments.
- If you add a package, name it, say why, and confirm it is a real, well-maintained package (not a look-
alike name).
- Never print real secret values; refer to them by variable name.
- If something lives outside the code (dashboards, hosting settings), give me exact click-by-click steps
instead.

FINISH WITH
1. Findings table: issue · file:line · severity · status (Fixed / Needs my action / Not applicable)
2. Files changed
3. How I can verify it myself in under 5 minutes
```

DONE WHEN

- Debug mode is off in production via environment config.
- Errors show a generic message plus a reference ID, never stack traces.
- API docs/GraphQL playgrounds are off or protected in production.
- Source maps aren't publicly served.

WATCH OUT FOR

- **Preview/staging deployments** are often public with debug on. Protect them with a password or platform auth.
- **Returning** `error.message` from a catch block is a sneaky leak. Database errors often include table and column names.

21

DEPLOYMENT & SUPPLY CHAIN | HIGH | 20–60 MIN

21/27 SECURITY PASS

Update vulnerable dependencies

No known, published vulnerability is sitting in your app waiting to be used.

WHAT THIS MEANS

Most of your app is other people's code: frameworks, libraries and their dependencies. When a vulnerability is found, it's published with the affected versions, and attackers scan the internet for apps still running them. Updating is the fix. Also make sure every package is real: AI tools sometimes invent package names, and attackers register those names with malware ("slopsquatting").

WHAT GOES WRONG

In the video, `image-lib 2.1.3` has a known hole and gets patched to 2.1.4. Without the update, anyone can look up the exploit for that exact version and run it against your app.

2-MINUTE SELF-CHECK

- Run `npm audit --omit=dev` (or `pnpm audit --prod`, `yarn npm audit`), `pip-audit`, or `osv-scanner` on your project.
- On GitHub, open **Security → Dependabot alerts** and enable Dependabot if it's off.
- Check your framework (Next.js, React, Django…) is on a currently supported, patched version.

CLAUDE PROMPT · CHECK 21

Paste your Project Context first, then this

```
Act as a senior application security engineer. Find and fix known-vulnerable dependencies, and check the dependency list for suspicious packages.

AUDIT
1. Run the right audit tools for the stack (npm audit / pnpm audit / yarn npm audit, pip-audit, bundle audit, cargo audit, composer audit, or osv-scanner for anything). If you can't run them, tell me exactly what to run and paste back.
2. Make a table: package · installed version · fixed version · severity · direct or transitive · used in production or dev-only · what it's used for.
3. Check the core framework and runtime (Node.js, Python, Next.js, React, Django, etc.) are on supported, patched versions.
4. Supply-chain sanity check for every DIRECT dependency: does the package really exist on the official registry, is it the well-known one (not a typo-squat like a near-identical name), does it have a real repository and reasonable download numbers, and has it changed owners recently? Flag anything an AI may have invented or misnamed. Flag packages with install scripts (postinstall) that aren't expected.
5. Confirm a lockfile is committed and CI installs from it (npm ci, pnpm install --frozen-lockfile, pip with hashes or a lock tool).

FIX
- Upgrade production Critical/High issues first, using the smallest version bump that contains the fix.
- For major-version upgrades, read the changelog, list breaking changes that affect this code, and fix them.
- For vulnerable transitive dependencies, upgrade the parent or use overrides/resolutions, and explain why.
- Don't run "npm audit fix --force" blindly.
- Rebuild and run tests after each group of upgrades.
- Set up Dependabot or Renovate (with grouped updates) and GitHub security alerts.
```

```
REPORT
What you upgraded, what remains (with reasons, such as "no fix available" or "dev-only, not shipped"),
and anything suspicious in the dependency list.

HOW TO WORK
- Audit first, and show me what you found before making large changes.
- Fix Critical and High issues directly. Ask me before any change users would notice (logging people
out, blocking a feature, changing a flow).
- Fix the root cause with the smallest safe change. Don't remove features, weaken other protections, or
leave "TODO: secure later" comments.
- If you add a package, name it, say why, and confirm it is a real, well-maintained package (not a look-
alike name).
- Never print real secret values; refer to them by variable name.
- If something lives outside the code (dashboards, hosting settings), give me exact click-by-click steps
instead.

FINISH WITH
1. Findings table: issue · file:line · severity · status (Fixed / Needs my action / Not applicable)
2. Files changed
3. How I can verify it myself in under 5 minutes
```

DONE WHEN

- No Critical/High vulnerabilities in production dependencies (or each one is explained).
- The framework is on a supported, patched version.
- Every direct dependency is a real, legitimate package.
- Lockfile committed; Dependabot/Renovate enabled.
- The app builds and tests pass after upgrades.

WATCH OUT FOR

- **Hallucinated packages are a real attack.** Before installing anything an AI suggests, check it on npmjs.com / pypi.org.
- **Frameworks ship urgent security patches** regularly. Subscribe to your framework's security announcements.

22

DEPLOYMENT & SUPPLY CHAIN | MEDIUM | 15–30 MIN

22/27 SECURITY PASS

Remove unused packages

Less code shipped means fewer things that can break, leak or be exploited.

WHAT THIS MEANS

Vibe-coded projects pile up packages, files and routes from experiments that were later abandoned. Each unused dependency is still code that can carry vulnerabilities or be hijacked, and each forgotten endpoint is still a door into your app. Removing them shrinks your attack surface (and your bundle).

WHAT GOES WRONG

Plantly installed `old-slider` and `csv-tools` for features that were dropped. Neither is used, but both still ship, and one gets a critical vulnerability next month. Meanwhile an old `/api/import-legacy` route from a deleted feature still works and has no auth. In the video, the app "still builds and runs" after cleanup.

2-MINUTE SELF-CHECK

- JavaScript/TypeScript: run `npx knip` or `npx depcheck`.
- Python: run `deptry .`
- Skim your API routes folder. Do you recognise and still use every route?

> CLAUDE PROMPT · CHECK 22

Paste your Project Context first, then this

```
Act as a senior application security engineer. Reduce this app's attack surface by removing what it doesn't use.

AUDIT
1. Find unused dependencies with the right tool (knip or depcheck for JS/TS, deptry for Python, or careful manual search). For each candidate, double-check it is truly unused: search for dynamic imports, usage in config files (build tools, PostCSS, Tailwind, Babel, ESLint, test setup), CLI usage in package.json scripts, peer dependencies and type packages.
2. Find dependencies that are in production dependencies but only needed in development (types, linters, test tools, build tools).
3. Find dead code that's still deployed: unused API routes and server actions, pages for removed features, old webhook endpoints, test/demo/seed routes, abandoned admin tools, and unused files in public/.
4. Find unused environment variables, third-party scripts (old analytics, chat widgets), OAuth apps and API keys tied to removed features.

FIX
- Remove unused dependencies and move dev-only ones to devDependencies (or the equivalent).
- Delete dead routes, pages and files. If unsure, list them for me instead of deleting.
- Remove unused third-party scripts and env vars.
- Rebuild, run the tests, and start the app to confirm everything still works.

REPORT
Table: item · type (package/route/file/script/env var/key) · evidence it's unused · action taken. List keys or OAuth apps I should revoke in their dashboards.

HOW TO WORK
- Audit first, and show me what you found before making large changes.
- Fix Critical and High issues directly. Ask me before any change users would notice (logging people out, blocking a feature, changing a flow).
- Fix the root cause with the smallest safe change. Don't remove features, weaken other protections, or leave "TODO: secure later" comments.
```

```
- If you add a package, name it, say why, and confirm it is a real, well-maintained package (not a look-alike name).
- Never print real secret values; refer to them by variable name.
- If something lives outside the code (dashboards, hosting settings), give me exact click-by-click steps instead.

FINISH WITH
1. Findings table: issue · file:line · severity · status (Fixed / Needs my action / Not applicable)
2. Files changed
3. How I can verify it myself in under 5 minutes
```

DONE WHEN

- No unused dependencies remain; dev tools are in devDependencies.
- Dead routes, pages and test endpoints are deleted.
- Unused third-party scripts and keys are removed or revoked.
- The app still builds and runs.

WATCH OUT FOR

- **Don’t trust a tool blindly.** Unused-dependency detectors have false positives, so always build and test after removing.

23

DEPLOYMENT & SUPPLY CHAIN | CRITICAL | 10–20 MIN

23/27 SECURITY PASS

Check for exposed files

Nothing private (env files, Git data, backups, database dumps) can be downloaded from your site.

WHAT THIS MEANS

Web servers serve whatever is in the public folder, and build steps sometimes copy more than you expect. Attackers run automated scanners that request paths like `/.env`, `/.git/config` and `/backup.zip` on millions of sites every day. If any of them returns a file, it's game over.

WHAT GOES WRONG

In the video, Plantly's public files include `.env.backup`, `db-dump.sql` and `.git/config`. The fix: each one now returns `404 · not public anymore`. One exposed `.env` leaks every key; an exposed `.git` folder lets anyone download your entire source code and its history.

2-MINUTE SELF-CHECK

- Run this against your own live domain; every line should say 404 (or 403):

```
for p in .env .env.local .env.production .env.backup .git/config .git/HEAD .DS_Store backup.zip
db.sql dump.sql database.sqlite; do echo "$p $(curl -s -o /dev/null -w '%{http_code}'
https://YOURAPP/$p)"; done
```

- Look inside your `public/`, `static/` and build output folders for anything that isn't meant for the public.

> CLAUDE PROMPT · CHECK 23

Paste your Project Context first, then this

```
Act as a senior application security engineer. Make sure no sensitive file can be downloaded from this
app's public URLs.

AUDIT
1. Inspect public/, static/, assets/, the build output (dist/, build/, out/, .next/static) and anything
else the server or host serves directly. List any file that shouldn't be public: .env files and backups,
.git, database files and dumps (.sql, .sqlite, .db), archives (.zip, .tar.gz), logs, IDE folders
(.vscode, .idea), .DS_Store, config files containing internal details, private documents, and test
fixtures with real data.
2. Check the server/host config for directory listing, and for rules that might serve dotfiles.
3. Check robots.txt and sitemap.xml don't advertise private paths (robots.txt is public and is NOT
protection).
4. Check cloud storage buckets (S3, R2, GCS, Supabase Storage, Firebase Storage): which are public, and
can their contents be listed?
5. Check .dockerignore / .vercelignore / similar so secrets aren't copied into images or deployments.

FIX
- Delete or move sensitive files out of public locations.
- Add deny rules so dotfiles and backup/dump extensions always return 404 (host config, nginx location
rules, or middleware), and disable directory listing.
- Make buckets private unless they must be public, and disable listing on public ones.
- If an exposed file contained secrets, add them to the rotation list for Check 25.

DELIVER
A script, scripts/check-exposed.sh, that takes my domain as an argument and requests these paths,
printing the status code for each and highlighting anything that returns 200:
/.env /.env.local /.env.production /.env.development /.env.backup /.env.bak /.env.old /.git/config
/.git/HEAD /.gitignore /.DS_Store /.vscode/settings.json /.idea/workspace.xml /backup.zip /backup.tar.gz
```

```
/db.sql /dump.sql /database.sql /database.sqlite /data.db /config.json /config.yml /docker-compose.yml
/.npmrc /phpinfo.php /server-status /swagger.json /openapi.json /graphql
plus any .map files under the static assets folder.
Only run it against my own domain.

HOW TO WORK
- Audit first, and show me what you found before making large changes.
- Fix Critical and High issues directly. Ask me before any change users would notice (logging people
out, blocking a feature, changing a flow).
- Fix the root cause with the smallest safe change. Don't remove features, weaken other protections, or
leave "TODO: secure later" comments.
- If you add a package, name it, say why, and confirm it is a real, well-maintained package (not a look-
alike name).
- Never print real secret values; refer to them by variable name.
- If something lives outside the code (dashboards, hosting settings), give me exact click-by-click steps
instead.

FINISH WITH
1. Findings table: issue · file:line · severity · status (Fixed / Needs my action / Not applicable)
2. Files changed
3. How I can verify it myself in under 5 minutes
```

DONE WHEN

- Every sensitive path returns 404/403 on the live site.
- Public folders contain only files meant for the public.
- Directory listing is off; buckets are private or not listable.
- A re-runnable `check-exposed.sh` script exists.

WATCH OUT FOR

- **Re-run this after every major deploy.** A new build step can re-introduce files.
- **If `.env` was ever publicly reachable,** treat every key in it as compromised (Check 25).

24

SECRETS & KEYS | CRITICAL | 15–30 MIN

24/27 SECURITY PASS

Scan my Git history for leaked secrets

Deleting a secret in a new commit doesn't remove it. Git remembers everything.

WHAT THIS MEANS

Every commit is saved forever. If you committed a key once, even for five minutes, even if you deleted it in the next commit, it's still in the history and visible to anyone with access to the repo (everyone, if it's public). Bots scan public GitHub for fresh keys within minutes of a push.

WHAT GOES WRONG

In the video, the latest `config.js` looks clean ("No key"). But a commit from two weeks ago, "add config", added `API_KEY = "sk_demo_51f…"`. Anyone who browses the history finds it.

2-MINUTE SELF-CHECK

- Install **gitleaks** and run it in your repo: `gitleaks git -v` (older versions: `gitleaks detect -v`).
- Or **TruffleHog**: `trufflehog git file://. --only-verified`
- On GitHub: Settings → Code security → enable **Secret scanning** and **Push protection**.

> CLAUDE PROMPT · CHECK 24

Paste your Project Context first, then this

```
Act as a senior application security engineer. Find any secret that has ever been committed to this repository and stop it from happening again.

AUDIT
1. Scan the FULL history, all branches and tags, with gitleaks (gitleaks git -v, or gitleaks detect -v on older versions) or TruffleHog (trufflehog git file://. ). If you can't install them, tell me the exact commands to run and paste back.
2. If no scanner is available, search history manually, e.g. git log --all -p -S "<pattern>" and git log --all --full-history -- "**/.env*", for patterns like: sk_live_, sk_test_, rk_live_, sk-, sk-ant-, AKIA, ghp_, github_pat_, xoxb-, xoxp-, AIza, -----BEGIN PRIVATE KEY-----, "service_role", postgres://…:…@, mongodb+srv://…:…@, api_key=, secret=, password=, and long base64/hex strings in config files. Include every .env file that was ever committed.
3. Report each finding as: type of secret · file · commit hash · date · branch · still present in the current code? Mask values: show only the first 4 characters.
4. Check other places secrets hide: committed notebooks, test fixtures, CI config files, Docker files, documentation and screenshots.

FIX
1. FIRST: every secret found goes on the rotation list (Check 25). Rewriting history does NOT make a leaked key safe; forks, clones and caches keep copies.
2. Remove any secrets still present in the current code and load them from environment variables.
3. Add prevention: a pre-commit hook that runs gitleaks (or the pre-commit framework with a secrets hook), a CI step that scans each push, and tell me how to enable GitHub secret scanning and push protection.
4. Only AFTER rotation, and only if I want it: explain how to purge the history with git filter-repo, the impact (rewritten commit hashes, force-push, every collaborator must re-clone), and how to ask GitHub to clear cached views if the repo is public.

Never print full secret values in your reply.

HOW TO WORK
- Audit first, and show me what you found before making large changes.
- Fix Critical and High issues directly. Ask me before any change users would notice (logging people
```

```
out, blocking a feature, changing a flow).
- Fix the root cause with the smallest safe change. Don't remove features, weaken other protections, or
leave "TODO: secure later" comments.
- If you add a package, name it, say why, and confirm it is a real, well-maintained package (not a look-
alike name).
- Never print real secret values; refer to them by variable name.
- If something lives outside the code (dashboards, hosting settings), give me exact click-by-click steps
instead.

FINISH WITH
1. Findings table: issue · file:line · severity · status (Fixed / Needs my action / Not applicable)
2. Files changed
3. How I can verify it myself in under 5 minutes
```

DONE WHEN

- The full history (all branches) has been scanned.
- Every secret found is on the rotation list.
- No secrets remain in the current code.
- Pre-commit scanning and GitHub push protection are on.

WATCH OUT FOR

- **Rotate first, rewrite later (if at all).** Rewriting history alone gives a false sense of safety.
- **Private repos aren’t safe storage.** They get shared, forked, cloned onto laptops and connected to third-party tools.

25

SECRETS & KEYS | CRITICAL | 20–60 MIN

25/27 SECURITY PASS

Revoke and replace compromised keys

Any key that was ever exposed is treated as stolen: replaced, deployed, then revoked.

WHAT THIS MEANS

If a key was ever in front-end code, Git history, a public file, logs, a screenshot, a shared chat or a support ticket, assume someone has it. Rotation means: create a new key, put it everywhere the app needs it, deploy, confirm it works, **then** revoke the old one and confirm it's dead. Only you can do the dashboard steps, so the prompt makes Claude your guide.

WHAT GOES WRONG

Plantly deletes the leaked key from the code but never revokes it, and the attacker keeps using it for months. Or the old key is revoked before the new one is deployed, and the app goes down. In the video, the old key now gets `401 · old key rejected` and the new key works.

2-MINUTE SELF-CHECK

- For each rotated key, call the provider's API with the **old** key. It must fail (401/403).
- Check each provider's usage/billing dashboard for activity you don't recognise during the time the key was exposed.

> CLAUDE PROMPT · CHECK 25

Paste your Project Context first, then this

```
Act as a senior application security engineer and incident responder. Help me safely rotate every
credential that may have been exposed. I will do the dashboard steps myself; you guide me and update the
code and config.

STEP 1: INVENTORY
List every credential the app uses: third-party API keys (AI, payments, email, SMS, maps), database
passwords and connection strings, Supabase service_role/secret keys, JWT/session signing secrets, OAuth
client secrets, webhook signing secrets, cloud access keys and service-account files, deploy/CI tokens,
SMTP credentials. For each: where it's stored (hosting env vars per environment, CI, local .env, other
services) and whether there's evidence it was exposed (client bundle, Git history, public file, logs;
use the findings from Checks 02, 11, 23 and 24).

STEP 2: PRIORITISE
Rank exposed keys by blast radius: payments and cloud/infrastructure first, then database and
service_role, then AI and messaging (cost abuse), then the rest.

STEP 3: A RUNBOOK PER KEY, in this order
a) Where to create the new key (the provider dashboard path), with the minimum permissions/scopes it
needs, plus restrictions (allowed IPs/domains) and spending limits where available.
b) Every place to update it (each environment in my host, CI secrets, local .env, other services or
teammates).
c) Redeploy and confirm the app works with the new key.
d) Revoke/delete the old key.
e) Confirm the old key now fails (give me a safe, read-only test request).
f) Check the provider's logs/usage for misuse during the exposure window, and what to look for.
g) Side effects to expect: rotating a JWT/session secret logs everyone out; rotating a database password
requires updating every service that connects; rotating a webhook secret requires updating the endpoint
config at the provider.

STEP 4: HARDEN
Update code to read every credential from environment variables, remove any fallback values, and add a
```

```
SECRETS.md (names, purpose, owner, where stored, last rotated; NEVER the values).

Never ask me to paste secret values into this chat, and never print them.

HOW TO WORK
- Audit first, and show me what you found before making large changes.
- Fix Critical and High issues directly. Ask me before any change users would notice (logging people
out, blocking a feature, changing a flow).
- Fix the root cause with the smallest safe change. Don't remove features, weaken other protections, or
leave "TODO: secure later" comments.
- If you add a package, name it, say why, and confirm it is a real, well-maintained package (not a look-
alike name).
- Never print real secret values; refer to them by variable name.
- If something lives outside the code (dashboards, hosting settings), give me exact click-by-click steps
instead.

FINISH WITH
1. Findings table: issue · file:line · severity · status (Fixed / Needs my action / Not applicable)
2. Files changed
3. How I can verify it myself in under 5 minutes
```

DONE WHEN

- Every exposed key has been replaced, deployed and revoked.
- Old keys are confirmed dead (401/403).
- Provider logs were checked for misuse.
- New keys have minimum scopes, restrictions and spending limits.
- A `SECRETS.md` inventory exists (without values).

WATCH OUT FOR

- **Deploy the new key before revoking the old one**, or you'll cause your own outage.
- **Never paste live secrets into any AI chat**, including this one. Refer to them by name.

26

MONITORING & RESPONSE | REVIEW | 45–90 MIN

26/27 SECURITY PASS

Run a thorough security review

One full pass across the whole app, like an attacker would see it.

WHAT THIS MEANS

Checks 01–25 each look at one area. This review looks at everything together: auth, secrets, data, inputs, files, payments, network and packages, the eight areas in the video. It also catches what individual checklists miss, especially **business-logic flaws** unique to your app: things like reusing a free trial, inviting yourself into someone else's team, or skipping a step in a flow.

WHAT GOES WRONG

Every individual check passes, but Plantly's "invite a teammate" endpoint lets anyone add themselves to any team by guessing its ID, and the payment webhook isn't verified. In the video, the review surfaces "Needs attention: payment webhook".

BEFORE YOU RUN IT

- Run this **after** working through the earlier checks, in a fresh session with full codebase access.
- Use the most capable model you have access to, and give it time; this is a big task.

> CLAUDE PROMPT · CHECK 26

Paste your Project Context first, then this

```
Act as a senior penetration tester and application security engineer doing a full pre-launch security review of this app. Think like an attacker. Do NOT change any code in this step; produce a review report first.

PHASE 1: MAP THE ATTACK SURFACE
Read the codebase (and SECURITY_MAP.md if it exists). Summarise: stack, hosting, data stored (and how sensitive), user roles, every entry point (pages, API routes, server actions, functions, webhooks, cron jobs, file uploads, admin tools), third-party services, and trust boundaries (what the browser controls vs. what the server decides).

PHASE 2: REVIEW EACH AREA
For each area, list what you checked and what you found:
1. Auth: server-side checks on every endpoint, session handling, password reset, OAuth/redirects, MFA for admins.
2. Access control: IDOR on every ID-based endpoint, admin routes, multi-tenant/team isolation, role changes.
3. Secrets: client bundle, env vars, Git history, logs, exposed files.
4. Data: database rules/RLS, least privilege, sensitive fields in responses, backups.
5. Inputs: server-side validation, mass assignment, SQL/NoSQL/command injection, XSS (including AI output), open redirects, SSRF in any feature that fetches a URL, email header injection.
6. Files: upload type/size/content, storage privacy, serving.
7. Payments: server-side pricing, webhook verification, idempotency, entitlements, coupons, race conditions.
8. Network: HTTPS/HSTS, CORS, security headers, cookies, CSRF, rate limiting (especially on paid/AI endpoints).
9. Packages and deployment: vulnerable or suspicious dependencies, debug mode, error leakage, source maps, preview deployments.
10. Business logic: abuse of free trials, referrals, credits, invites, quotas, state transitions (can steps be skipped or repeated?), race conditions, and anything unique to this app.
11. AI features (if any): prompt injection through user content or documents, tools/actions the model can trigger, data leaking between users, cost abuse.
```

```
PHASE 3: REPORT
Write SECURITY_REVIEW.md with:
- An executive summary in plain English (5 lines max).
- A findings table sorted by severity: ID · title · severity (Critical/High/Medium/Low) · location
(file:line or endpoint) · how an attacker would exploit it, step by step · recommended fix · effort
(S/M/L).
- What you checked and found no problems with, so I know what was covered.
- Anything you could NOT verify from the code (dashboard settings, hosting config), with exact steps for
me to check.
Rate severity honestly. Don't inflate minor issues or hide real ones, and say how confident you are in
each finding.

Then STOP and wait for my go-ahead. After I approve, fix the findings in severity order, one at a time,
and update the table's status as you go.
```

DONE WHEN

- `SECURITY_REVIEW.md` exists with a ranked findings table.
- Every Critical and High finding is fixed or has a clear owner and date.
- The “couldn’t verify” items have been checked by you manually.

WATCH OUT FOR

- **AI reviewers can miss things and can invent problems.** Ask “show me the exact code that proves this” for any finding you’re unsure about.
- **Run it again** after big features, and before every major launch.

27

MONITORING & RESPONSE | VERIFY | 45–90 MIN

27/27 SECURITY PASS

Test the protections and give me a report

Every fix is proven by a test that tries to break it, and summarised in one report.

WHAT THIS MEANS

A fix you haven't tested is a guess. This final step turns every protection into an automated test that actively tries to break it: signed-out requests, cross-user access, tampered prices, expired tokens, oversize uploads, brute force. If a test fails, Claude fixes the code and re-tests. The result is a security report you can actually trust, and tests that keep protecting you on every future deploy.

WHAT GOES WRONG

Plantly "added" admin protection, but the role check only ran in the page layout, not in the API. The test `Admin route blocks users` fails, the fix ("role check on the server") is applied, and the re-test passes. Without the test, that hole would have shipped.

BEFORE YOU RUN IT

- Run against **local or staging**, never destructive tests against live production or real customer data.
- Keep the generated tests in your repo and run them in CI on every push.

> CLAUDE PROMPT · CHECK 27

Paste your Project Context first, then this

```
Act as a senior application security engineer and QA lead. Prove that this app's security protections
actually work, fix anything that doesn't, and give me a final report.

RULES
- Run tests only against localhost or a staging environment, never destructive tests against production,
and never with real customer data. Use dedicated test accounts (at least: signed-out visitor, user A,
user B, admin).
- Write the tests as real, repeatable automated tests in the project's existing test framework (e.g.
Vitest/Jest + supertest, Playwright, pytest + httpx). If no test setup exists, create a minimal one and
explain how to run it.

STEP 1: WRITE AND RUN "TRY TO BREAK IT" TESTS for every protection that applies to this app:
01 Auth: every protected endpoint returns 401 when signed out.
03 Data: user A can't read/update/delete user B's resources (403/404); the public key alone reads
nothing private.
04 HTTPS: HTTP redirects to HTTPS; HSTS present (staging/production URL only).
05 Passwords: stored hashes use Argon2id/bcrypt/scrypt; no password in logs or responses.
06 Validation: negative/oversized/wrong-type/extra-field inputs are rejected; "role":"admin" is ignored.
07 XSS: payloads render as text.
08 Logging: a failed login produces a log event without secrets.
09 Admin: a normal user gets 403 on every admin page and endpoint.
10 Injection: injection payloads return normal empty/400 responses.
11/02 Secrets: the production build contains no secret patterns.
12 Rate limits: the Nth+1 request returns 429.
13 CORS: an unknown origin gets no Allow-Origin header.
14 Headers: all required security headers are present.
15 Cookies: session cookies have HttpOnly, Secure and SameSite; a logged-out session is rejected.
16 CSRF: a state-changing request with a foreign Origin/no token is rejected.
17 Reset links: used, expired and superseded tokens fail; real and fake emails get identical responses.
18 Uploads: a disguised HTML file, a script SVG, an oversized file and a ../ filename are rejected.
19 Payments: a tampered price, a negative quantity, a reused coupon, the success URL without payment,
```

```
and an unsigned or duplicate webhook all fail safely.
20 Debug: error responses contain no stack traces, paths or queries.
23 Exposed files: sensitive paths return 404.
25 Keys: old keys are rejected (I'll run these manually if needed; give me the commands).

STEP 2: FIX AND RE-TEST
For every failing test: explain the cause, apply the smallest correct fix, and re-run until it passes.
Don't weaken or delete a test to make it pass.

STEP 3: REPORT
Write SECURITY_REPORT.md containing:
1. Summary: total checks, Passed, Fixed and re-tested, Still open, Not applicable.
2. A table of all 27 checks: # · check · status (Passed / Fixed & re-tested / Open – next step / N/A) ·
evidence (test name or command) · notes.
3. Findings and fixes: what was wrong, what changed (files), and the test that now proves it.
4. Open items, with risk level and the exact next step for each.
5. Manual actions only I can do (dashboard settings, key rotation, MFA, billing caps), as a checklist.
6. How to run the security tests, and a recommendation to run them in CI on every push.
```

DONE WHEN

- Automated tests exist for every protection that applies to your app.
- All tests pass, or each failure is listed as an open item.
- `SECURITY_REPORT.md` summarises findings, fixes and open items.
- Security tests run in CI on every push.

WATCH OUT FOR

- **Watch for “fixing” by weakening the test.** If a test changes, ask why.
- **Green tests ≠ perfectly secure.** They prove what they test. Re-run the review (Check 26) as the app grows.

BEYOND THE VIDEO

Bonus checks: the ones people forget

Seven extra checks that matter especially for AI-built apps: AI feature abuse, internal-network attacks, backups, your own accounts, surprise bills, privacy and email spoofing.

B1 Secure your AI features HIGH

If your app calls an LLM, users can try to hijack it with prompt injection ("ignore your instructions and..."), extract your system prompt, make it trigger actions it shouldn't, or simply run up your bill. Text inside uploaded documents or web pages can carry the same attacks.

> CLAUDE PROMPT · B1 — Paste your Project Context first

```
Act as an AI application security engineer. Review every feature in this app that calls an LLM.
1. List each AI call: model, where the API key lives (must be server-only), what user content and which documents/data go into the prompt, and what the output is used for.
2. Prompt injection: assume any user text, file, email or web page fed to the model may contain hostile instructions. Make sure the model cannot reveal secrets (no keys or other users' data in prompts), cannot access data the current user isn't allowed to see, and cannot trigger tools/actions beyond that user's permissions. Destructive or costly actions need explicit user confirmation.
3. Treat model output as untrusted: sanitise before rendering as HTML/Markdown (Check 07), validate before using it in queries, URLs, code or commands.
4. Cost abuse: authentication required, per-user rate limits and daily quotas, maximum input and output tokens, and a provider-side monthly spending cap.
5. Privacy: don't send more personal data to the model than needed; log prompts carefully or not at all.
Fix the issues and list what you changed.
```

B2 Block SSRF in URL-fetching features HIGH

Any feature that fetches a URL a user provides (link previews, "import from URL", webhooks you call, image-from-URL) can be pointed at your own internal network or the cloud metadata service, which can expose credentials.

> CLAUDE PROMPT · B2 — Paste your Project Context first

```
Act as a senior application security engineer. Find every place the server fetches a URL that a user can influence (link previews, imports, image fetching, outgoing webhooks, PDF/screenshot generation, AI tools that browse).
For each: allow only http/https; resolve the hostname and block private, loopback, link-local and internal ranges (127.0.0.0/8, 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, 169.254.0.0/16 including the cloud metadata IP 169.254.169.254, ::1, fc00::/7) for the initial request AND every redirect; use an allowlist of domains where possible; set short timeouts and a maximum response size; never return raw internal error details. Use a maintained SSRF-protection library if one fits the stack. Add tests with internal addresses.
```

B3 Backups you can actually restore HIGH

Security isn't only about stopping attackers. A bad migration, a buggy AI edit or a deleted table can wipe your data. Backups only count if you've tested restoring them.

> CLAUDE PROMPT · B3 — Paste your Project Context first

```
Act as a senior reliability and security engineer. Review this app's backup and recovery setup.
1. What is backed up (database, file storage, configuration), how often, where backups are stored, and for how long? Is point-in-time recovery available on my database plan?
2. Are backups protected from the same credentials as production (so an attacker or a mistake can't delete both)?
3. Write a step-by-step RESTORE.md for restoring the database and files to a new environment, and tell me how to run a test restore without touching production.
4. Recommend the minimum setup for my stack and plan, with the exact dashboard steps.
```

B4 Lock down your own accounts CRITICAL

The easiest way into your app is often through *you*: your GitHub, hosting, domain registrar, database dashboard, Stripe or email. One phished password there bypasses every check in this toolkit.

> CLAUDE PROMPT · B4 — Paste your Project Context first

```
Act as a security advisor. Help me secure the accounts that control this app. Based on the codebase and config, list every service I depend on (code hosting, hosting/deploy, DNS and domain registrar, database, auth provider, payments, email, AI providers, analytics, error tracking, app stores).
For each, give me a checklist: turn on MFA (an authenticator app or passkey, not SMS where possible); review team members and remove anyone who no longer needs access; give least-privilege roles; review connected OAuth apps and integrations; review active API keys and deploy tokens; turn on login/security alerts; turn on domain lock at the registrar. Format it as a checklist I can tick off.
```

B5 Spending caps and abuse alarms HIGH

For vibe-coded apps, the most likely "attack" is a surprise bill: leaked AI keys, a loop hammering a paid API, SMS-pumping fraud, or a sudden traffic spike on usage-based hosting.

> CLAUDE PROMPT · B5 — Paste your Project Context first

```
Act as a cloud cost and abuse-prevention advisor. List every service this app uses that bills by usage (hosting, serverless, database, storage, bandwidth, AI/LLM APIs, email, SMS, maps, search). For each, tell me exactly how to set (1) a billing alert, (2) a hard spending cap or quota if the provider supports one, and (3) per-key usage limits. For SMS, restrict destination countries. Also add app-level per-user quotas on the most expensive features, and a simple daily usage summary in the logs.
```

B6 Privacy and data minimisation MEDIUM

Data you don't collect can't be leaked. Privacy laws (GDPR in the EU, India's DPDP Act, CCPA in California and others) may also require you to let users see, export and delete their data, and to notify people after a breach.

> CLAUDE PROMPT · B6 Paste your Project Context first

```
Act as a privacy-focused security engineer. Map every piece of personal data this app collects: where it's stored, who can see it, which third parties receive it (analytics, AI providers, email, error tracking), and how long it's kept.
Then: flag data we don't actually need; make sure personal data isn't sent to analytics or error tracking unnecessarily; implement (or plan) account deletion that removes or anonymises the user's data across the database, storage and third parties; implement a data export for users; and list what my privacy policy needs to mention based on what the app really does. Note which items need legal review.
```

B7 Protect your email domain MEDIUM

Without SPF, DKIM and DMARC records, anyone can send emails that look like they come from your domain: perfect for phishing your users with fake "reset your password" messages. These records also keep your real emails out of spam.

> CLAUDE PROMPT · B7 Paste your Project Context first

```
Act as an email security advisor. Based on the email provider(s) this app uses (transactional and personal), tell me exactly which DNS records to add for my domain: SPF (one record listing all senders), DKIM (from each provider), and DMARC (start with p=none and a reporting address, then move to quarantine/reject once reports are clean). Also check that the app's emails never include sensitive data unnecessarily and that links in emails use my configured APP_URL. Give me the steps to verify everything with a free checker.
```

KEEP THIS HANDY

Emergency card: if something goes wrong

A leaked key, strange charges, users reporting accounts they didn't touch, data that changed on its own. Here's what to do, in order.

1 Stay calm, keep evidence

Don't delete logs, users or the repo. Write down what you noticed and when.

2 Stop the bleeding

Rotate exposed keys (Check 25), payments, cloud and database first. Put the affected feature into maintenance mode if needed.

3 Kill sessions

Rotate session/JWT secrets or revoke sessions so attackers are logged out. Force password resets if accounts were affected.

4 Investigate

Check app logs, database logs and provider dashboards: when it started, what was accessed, which accounts.

5 Close the hole

Fix the vulnerability, add a test that proves it's fixed (Check 27), deploy.

6 Check your data

Was data read, changed or deleted? Are your backups intact (Bonus B3)?

7 Tell people

If users' personal data was affected, tell them honestly. Depending on where you and your users are, laws like GDPR (72-hour regulator notice) or India's DPDP Act may require formal notification. Get legal advice.

8 Learn

Write a short post-mortem: cause, impact, fix, and what will stop it happening again.

> CLAUDE PROMPT · INCIDENT RESPONSE

Describe what you saw in the brackets

```
Act as an incident responder. I think my app has been compromised. Here's what I've noticed: [describe what you saw, when, and anything unusual].
Help me, in this order:
1. Containment: the first three actions to take in the next 15 minutes, with exact steps.
2. Which credentials to rotate first, and how, without taking the app down unnecessarily.
3. Where to look in the code, logs and provider dashboards to understand what happened, and the exact queries/filters to use.
4. The most likely vulnerability given the codebase, and the fix plus a test.
5. A plain-English summary I could adapt to tell affected users.
Don't print secret values. Ask me before running anything destructive.
```

Leaked an API key right now?

Go to that provider's dashboard and revoke it immediately, even before you've deployed a replacement, if it's a payment, cloud or AI key. A few minutes of downtime is cheaper than the alternative. Then follow Check 25.

AFTER LAUNCH

Keep it secure as you keep shipping

Security isn't a one-time pass. Every new feature you vibe-code can reopen an old hole. Use these two tables to stay ahead with very little effort.

When you add… re-run these checks

YOU'RE ADDING	CHECKS
Login or sign-up	`01, 05, 12, 15, 16, 17`
A new database table	`03, 06, 10`
A new API route / server action	`01, 03, 06, 09, 12`
Comments, profiles, any user text	`06, 07, 12`
Payments or subscriptions	`06, 12, 19, 08`
File uploads	`18, 03, 12`
A new third-party API / key	`02, 11, B5`
An admin tool	`09, 08, 07`
AI / LLM features	`02, 07, 12, B1, B5`
Fetching user-supplied URLs	`B2, 06`
A new package	`21 (check it's real first)`
Before every launch	`23, 20, 26, 27`

Your maintenance rhythm

WHEN	WHAT
Every commit	Pre-commit secret scan runs automatically (Check 24).
Every push / deploy	Security tests run in CI (Check 27). Re-run check-exposed.sh after big deploys (Check 23).
Weekly	Review Dependabot/Renovate PRs and security alerts. Glance at failed-login and 5xx alerts.
Monthly	Run dependency audit (21) and remove unused packages (22). Review admin users and team access (B4). Check billing dashboards (B5).
Quarterly	Full security review (26) and test run (27). Test a backup restore (B3). Rotate high-value keys.
Every big feature	Run the checks in the table on the left before shipping.

Make Claude remember your rules

Add a short security section to your project's AI rules file (`CLAUDE.md` , `.cursorrules` , or your tool's "project knowledge"), so new code is written safely from the start. For example: *"Every API route must call requireUser() and validate input with Zod. Never use secret keys in client code. Never trust prices from the client. All new tables need RLS policies. Ask before adding packages."*

WHAT "DONE" LOOKS LIKE

Your final security report

Check 27 ends with a `SECURITY_REPORT.md` in your project. Here's an illustrative example of the shape to expect, so you can tell at a glance whether you got a real report or a vague summary.

Security report · Plantly

Illustrative example

27	21	2	4
Checks run	Passed or fixed & re-tested	Still open	Not applicable

#	CHECK	STATUS	EVIDENCE / NEXT STEP
01	Set up proper authentication	Passed	auth.test.ts · 14 endpoints → 401
03	Secure my database	Fixed & re-tested	rls.test.ts · A ↛ B on 6 tables
09	Lock down my admin routes	Fixed & re-tested	admin.test.ts · role check on server
12	Add rate limiting	Passed	ratelimit.test.ts · 6th login → 429
19	Verify prices and payments	Open – next step	Webhook idempotency: add event-ID table
25	Revoke compromised keys	Needs my action	Rotate OpenAI key in dashboard
18	Restrict file uploads	N/A	No upload features

...plus sections for findings & fixes, open items with risk levels, manual actions only you can do, and how to run the security tests.

A GOOD REPORT HAS

- Every check listed, including N/A ones.
- Evidence for every "Passed": a test name or a command.
- Specific next steps for open items.
- A clear list of what *you* must do manually.

PUSH BACK IF YOU SEE

- "Everything looks secure!" with no evidence.
- Tests that were deleted or weakened to pass.
- Findings with no file, line or endpoint.
- Critical items quietly marked "N/A".

PLAIN-ENGLISH DICTIONARY

Glossary

The security words you'll see in Claude's answers, explained in one line each.

Authentication
Proving *who* a user is (login).

Authorization
Deciding *what* that user is allowed to do.

IDOR
Insecure Direct Object Reference: changing an ID to access someone else's data.

RLS
Row Level Security: database rules that limit which rows each user can see or change.

XSS
Cross-site scripting: attacker-controlled script running in your users' browsers.

CSRF
Cross-site request forgery: another site making a user's browser act on your app.

CORS
Browser rules for which other sites can read your API's responses.

SQL injection
User input changing the structure of a database query.

SSRF
Tricking your server into fetching internal or private URLs.

Hashing
One-way scrambling of passwords so they can be checked but not read.

Session / cookie
The token that keeps a user logged in. Whoever has it *is* the user.

JWT
A signed token carrying user info. It must be verified, and its contents are readable by anyone.

Rate limiting
Capping how many requests someone can make in a time window.

Webhook
A server-to-server notification (e.g. "payment succeeded"). It must be signature-verified.

CSP
Content-Security-Policy: a header that limits which scripts and resources a page may load.

HSTS
A header telling browsers to always use HTTPS for your site.

Publishable vs secret key
Publishable keys are designed to be public. Secret keys must never leave the server.

Rotation
Replacing a credential with a new one, then revoking the old one.

Slopsquatting
Attackers publishing malware under package names that AI tools tend to hallucinate.

Prompt injection
Hidden instructions in user content that try to hijack an AI model.

Ship fast. Ship safe.

You don't have to choose. Run the checks, keep the tests, and re-run the review whenever your app grows. Your users will never notice the security you added, and that's exactly the point.